DATA STRUCTURE AND APPLICATIONS

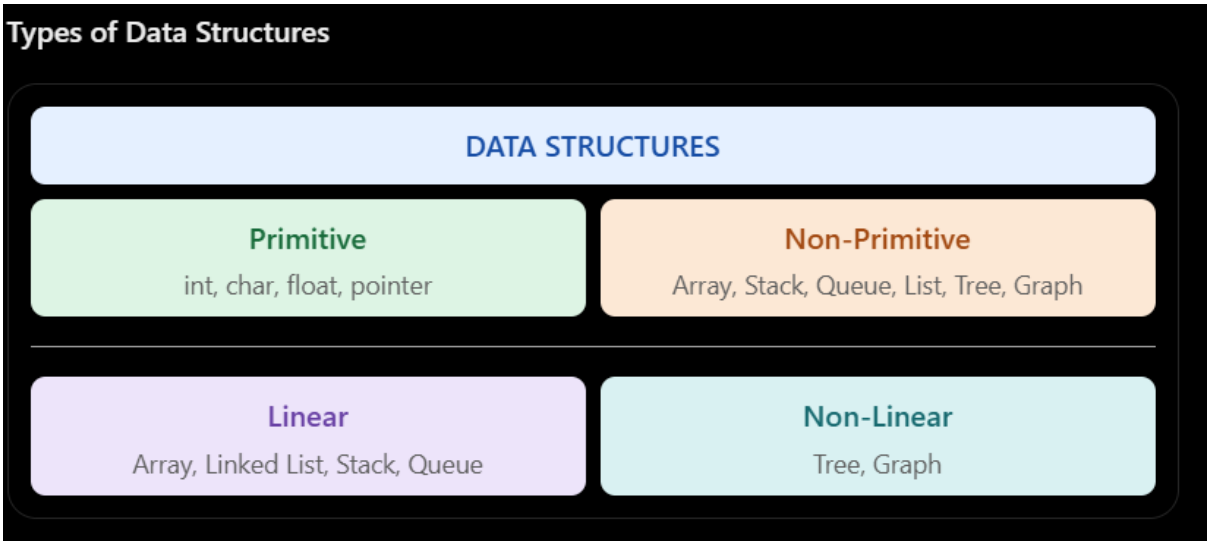
MODULE 1

1. Define Data Structures. List and explain the types of Data Structures.

Definition

A data structure is a way of organizing, storing, and managing data in a computer so that it can be accessed and modified efficiently.

It helps perform operations such as insertion, deletion, searching, sorting, and updating.



A. Primitive Data Structures

Primitive data structures are the basic built-in data types provided by a programming language.

Examples in C:

Data type	Description	Example
int	Stores integers	int age = 20;
char	Stores a character	char grade = 'A';
float	Stores decimal values	float x = 5.5;

double	Stores higher-precision decimal values	double d = 12.45;
Pointer	Stores a memory address	int *p;

B. Non-Primitive Data Structures

Non-primitive data structures are created using primitive data types to store and organize multiple values.

They are divided into two types:

1. Linear Data Structures

Elements are arranged sequentially, one after another.

- Array: Stores elements of the same type in contiguous memory locations.
- Linked List: Stores elements in nodes connected using pointers.
- Stack: Follows LIFO (Last In, First Out).
- Queue: Follows FIFO (First In, First Out).

2. Non-Linear Data Structures

Elements are arranged hierarchically or in network form.

- Tree: Represents hierarchical relationships, such as parent and child nodes.
- Graph: Represents vertices connected by edges.

Example

int marks[5] = {70, 80, 90, 60, 85};

Here, marks is an array containing five integer elements.

Advantages

1. Organizes data systematically.
2. Makes searching and sorting easier.
3. Improves data access and processing.
4. Helps manage large amounts of data.
5. Supports efficient memory utilization depending on the structure.

Disadvantages



1. Some structures require additional memory.
2. Incorrect selection can reduce performance.
3. Complex structures may be difficult to implement.
4. Certain operations may take more time depending on the structure.

Conclusion: Data structures are essential for organizing data and designing efficient programs.

2. Differentiate between Structure and Union in C.

Definition

Structure: A structure is a user-defined data type that groups variables of different data types under one name. Each member has its own memory location.

Union: A union is a user-defined data type that groups variables of different data types, but all members share the same memory location.

Difference between Structure and Union

No.	Structure	Union
1	Declared using struct.	Declared using union.
2	Each member has separate memory.	All members share memory.
3	Size is approximately the sum of member sizes plus padding.	Size is based on the largest member plus alignment padding.
4	Multiple members can store valid values simultaneously.	Only one member's value should be considered active at a time.
5	Changing one member does not normally affect another.	Writing one member may overwrite the representation of another.
6	Uses more memory when several members exist.	Often uses less memory.
7	All members can be initialized individually.	Typically one member is initialized at a time.

8	Suitable for records containing multiple fields.	Suitable when data can be represented in different alternative forms.
9	Members have distinct addresses within the structure.	Members begin at the same address.
10	Example: Student information.	Example: A value represented as an integer or float.

Structure Example

```
#include <stdio.h>

struct Student {
   int roll;
   float marks;
   char grade;
};

int main() {
   struct Student s = {1, 85.5, 'A'};

   printf("%d\n", s.roll);
   printf("%.1f\n", s.marks);
   printf("%c\n", s.grade);

   return 0;
}
```

Output:

```
1
```



85.5

A

Union Example

```
#include <stdio.h>

union Data {
   int i;
   float f;
   char ch;
};

int main() {
   union Data d;

   d.i = 10;
   printf("%d\n", d.i);

   d.f = 5.5;
   printf("%.1f\n", d.f);

   return 0;
}
```

Output:

10

5.5



Here, assigning d.f replaces the previously stored union representation. The earlier d.i value should not be relied upon.

Advantages of Structure

1. Stores multiple related values together.
2. Each member can maintain its own value.
3. Useful for student, employee, and product records.

Advantages of Union

1. Saves memory when only one alternative value is needed at a time.
2. Useful in memory-sensitive applications.
3. Supports different interpretations of the same storage.

Conclusion: Structures allocate separate storage for their members, while unions share storage among members.

3. Compare Primitive and Non-Primitive Data Structures.

Definition

Primitive data structures: Basic data types directly supported by a programming language, such as int, char, float, and double.

Non-primitive data structures: Data structures created using primitive data types to organize collections of data, such as arrays, linked lists, stacks, queues, trees, and graphs.

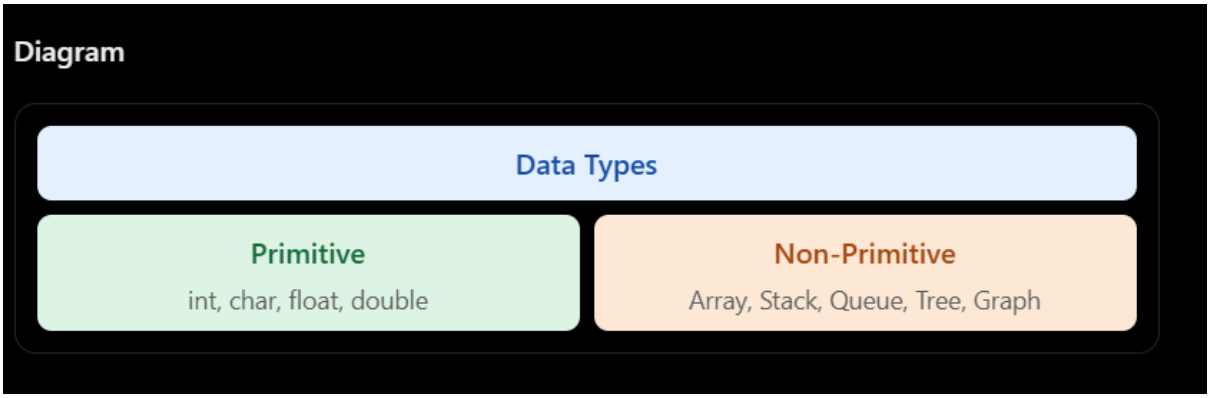
Difference

No.	Primitive	Non-Primitive
1	Basic data types.	Derived or user-defined structures.
2	Usually directly supported by the language.	Constructed using primitive types.
3	Generally stores a single value.	Can organize multiple values.
4	Usually simple to use.	May require algorithms to manage.
5	Generally uses less memory per value.	May need additional memory.



6	Usually has a fixed type and representation.	Organization depends on implementation.
7	Examples: int, char.	Examples: array, stack, tree.
8	Used for basic calculations and values.	Used for data organization and processing.
9	Usually does not require special operations.	May support insertion, deletion, traversal, etc.
10	Example: int age = 20;	Example: int marks[5];

Diagram



Advantages of Non-Primitive Structures

1. Organize large data collections.
2. Support efficient searching and sorting.
3. Represent complex relationships.
4. Can be designed for specific applications.

Disadvantages

1. Implementation may be complex.
2. Some structures require extra memory.
3. Performance depends on the selected structure and operations.

Conclusion: Primitive types store basic values, while non-primitive structures organize and manage collections of data.

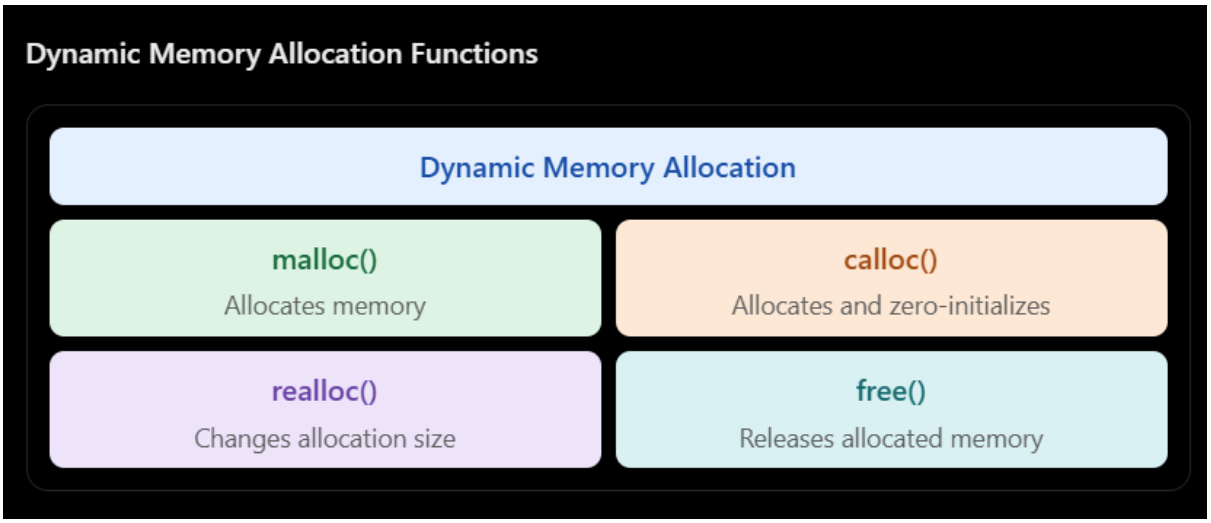
4. List and explain Dynamic Memory Allocation with examples.

Definition

Dynamic memory allocation (DMA) is the process of allocating memory during program execution, according to the program's requirements.

In C, dynamic memory allocation is performed using functions available in the stdlib.h header file.

Dynamic Memory Allocation Functions



1. malloc()

malloc() allocates a block of memory of the specified size.

Syntax:

ptr = malloc(size);

Example:

int *p;

p = malloc(5 * sizeof(int));

It allocates memory for five integers.

The allocated memory is uninitialized.

2. calloc()

calloc() allocates memory for multiple elements and initializes the allocated bytes to zero.

Syntax:

ptr = calloc(number, size);

Example:

int *p;

p = calloc(5, sizeof(int));

It allocates memory for five integers, with their initial values set to zero.

3. realloc()

realloc() changes the size of a previously allocated memory block.

Syntax:

ptr = realloc(ptr, new_size);

Example:

p = realloc(p, 10 * sizeof(int));

It requests a memory block large enough for ten integers. The block may move to a different location.

4. free()

free() releases dynamically allocated memory.

Syntax:

free(ptr);

Example:

free(p);

p = NULL;

Setting the pointer to NULL after releasing its memory helps avoid accidentally using the old address.

Complete C Program

#include <stdio.h>



```
#include <stdlib.h>

int main() {
   int *p;
   int i;

   p = malloc(3 * sizeof(int));

   if (p == NULL) {
      printf("Memory allocation failed");
      return 1;
   }

   p[0] = 10;
   p[1] = 20;
   p[2] = 30;

   printf("Initial values:\n");

   for (i = 0; i < 3; i++) {
      printf("%d ", p[i]);
   }

   p = realloc(p, 5 * sizeof(int));

   if (p == NULL) {
```



```
    printf("\nReallocation failed");
    return 1;
  }

  p[3] = 40;
  p[4] = 50;

  printf("\nAfter reallocation:\n");

  for (i = 0; i < 5; i++) {
    printf("%d ", p[i]);
  }

  free(p);
  p = NULL;

  return 0;
}
```

Output:

Initial values:

10 20 30

After reallocation:

10 20 30 40 50

Advantages

1. Memory can be allocated when required.



2. Supports variable-sized data.
3. Helps implement linked lists, trees, and graphs.
4. Avoids reserving unnecessarily large fixed arrays.

Disadvantages

1. Incorrect use may cause memory leaks.
2. Allocation can fail.
3. Accessing released memory is unsafe.
4. Dynamic allocation may introduce overhead.

Conclusion: Dynamic memory allocation provides flexible memory management during program execution.

5. Define Structure. Declare, initialize, and access structure variables in C with a suitable program.

Definition

A structure in C is a user-defined data type that groups variables of different data types under a single name.

Each member of a structure has its own storage.

Structure Declaration

Syntax:

```
struct structure_name {
   data_type member1;
   data_type member2;
};
```

Example:

```
struct Student {
   int roll;
   char name[20];
   float marks;
```



};

Structure Initialization

Structure variables can be initialized when they are declared.

struct Student s1 = {1, "Ravi", 85.5};

Accessing Structure Members

The dot (.) operator is used to access structure members.

s1.roll

s1.name

s1.marks

C Program

```
#include <stdio.h>

struct Student {
   int roll;
   char name[20];
   float marks;
};

int main() {
   struct Student s1 = {1, "Ravi", 85.5};

   printf("Student Details\n");

   printf("Roll Number: %d\n", s1.roll);
   printf("Name: %s\n", s1.name);
   printf("Marks: %.2f\n", s1.marks);
```



```
    return 0;
}
```

Output:

Student Details

Roll Number: 1

Name: Ravi

Marks: 85.50

Step-by-Step Working

1. struct Student defines a structure.
2. It contains roll number, name, and marks.
3. s1 creates a structure variable.
4. Values are assigned during initialization.
5. The dot operator accesses each member.
6. printf() displays the values.

Advantages

1. Groups related data together.
2. Supports different data types.
3. Improves program organization.
4. Useful for records such as students and employees.

Disadvantages

1. Structure members may require padding.
2. Large structures can occupy significant memory.
3. Accessing members requires the appropriate operator.

Conclusion: Structures provide a convenient way to store and manage related information in C.

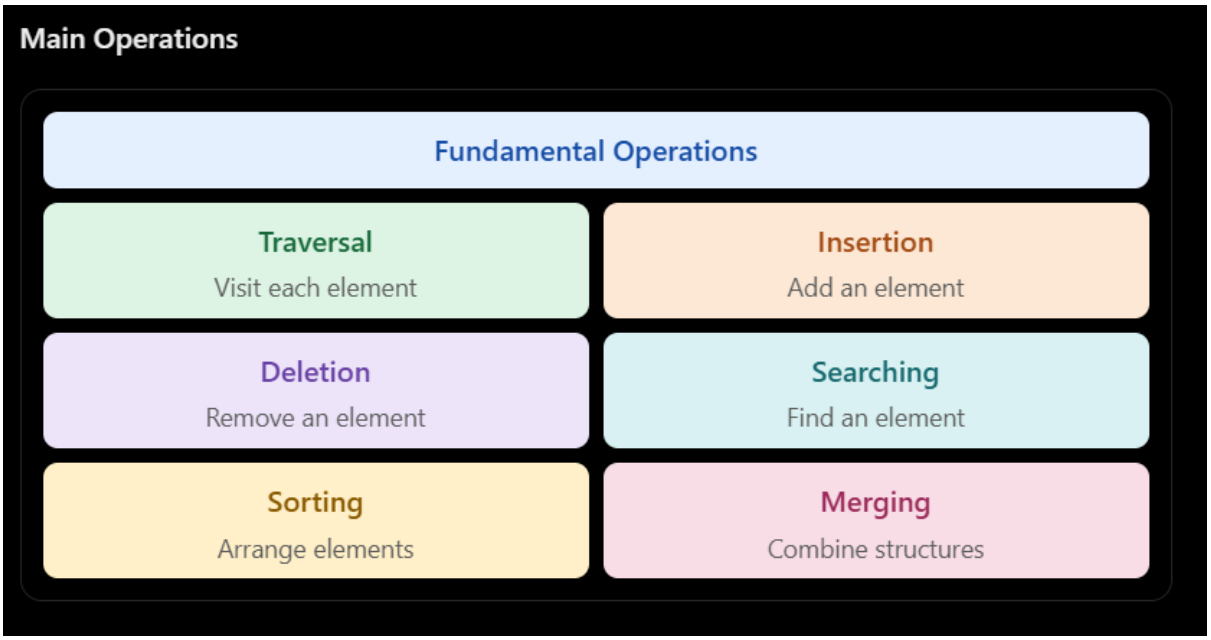


6. Demonstrate the fundamental operations performed on data structures with suitable examples.

Definition

Fundamental operations are the basic actions performed on a data structure to store, retrieve, modify, and process data.

Main Operations



1. Traversal

Traversal means visiting each element of a data structure.

Example:

Array: [10, 20, 30]

Traversal output:

10 → 20 → 30

2. Insertion

Insertion means adding a new element.

Example:

Before: 10 20 30

Insert: 25

After: 10 20 25 30

3. Deletion

Deletion means removing an element.

Example:

Before: 10 20 30

Delete: 20

After: 10 30

4. Searching

Searching finds the location of a particular element.

Example:

Array: 10 20 30 40

Search: 30

Result: Found at index 2

5. Sorting

Sorting arranges elements in ascending or descending order.

Example:

Before: 30 10 40 20

After: 10 20 30 40

6. Merging

Merging combines two data structures into one.

Example:

A = 10 20

B = 30 40

Merged = 10 20 30 40

Advantages

1. Makes data processing easier.
2. Supports efficient searching and modification.
3. Helps organize information.
4. Provides operations suitable for different applications.

Disadvantages

1. Some operations require shifting elements.
2. Searching may take longer for large structures.
3. Performance depends on the data structure.

Conclusion: Fundamental operations are necessary for manipulating and maintaining data structures.

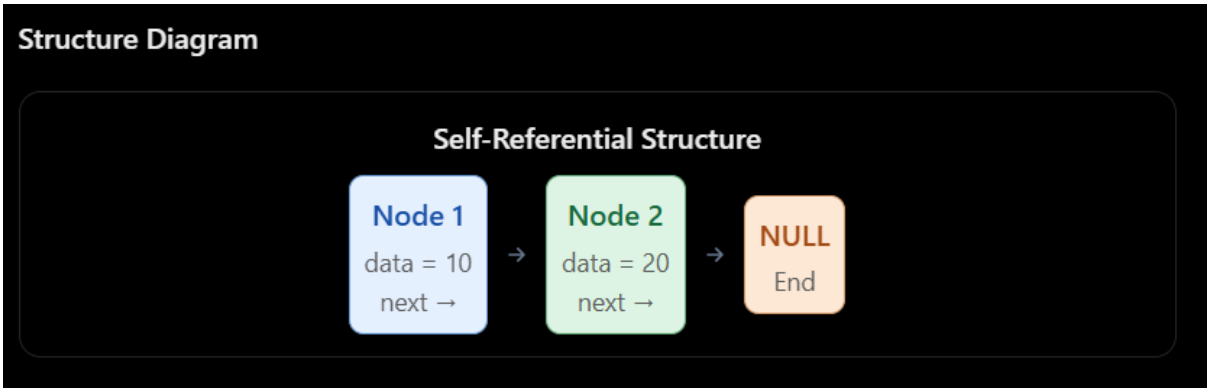
7. Examine the design and working of self-referential structures in C with an example.

Definition

A self-referential structure is a structure that contains a pointer to another structure of the same type.

It is commonly used to create linked lists, trees, and other dynamic data structures.

Structure Diagram



Declaration

```
struct Node {
   int data;
```

```
  struct Node *next;
};
```

Here:

- data stores the value.
- next is a pointer to another struct Node.
- NULL indicates that no next node exists.

C Program

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *next;
};

int main() {
  struct Node *first, *second;

  first = malloc(sizeof(struct Node));
  second = malloc(sizeof(struct Node));

  if (first == NULL || second == NULL) {
    free(first);
    free(second);
    return 1;
  }
```



```
    first->data = 10;
    first->next = second;

    second->data = 20;
    second->next = NULL;

    printf("%d\n", first->data);
    printf("%d\n", first->next->data);

    free(second);
    free(first);

    return 0;
}
```

Output:

10

20

Working

1. Define the structure Node.
2. Allocate memory for two nodes.
3. Store 10 in the first node.
4. Store the address of the second node in first->next.
5. Store 20 in the second node.
6. Set second->next = NULL.
7. Access the second node using first->next.



8. Release allocated memory.

Advantages

1. Supports dynamic memory allocation.
2. Allows linked data structures.
3. Nodes can be added or removed dynamically.
4. Memory need not be contiguous.

Disadvantages

1. Requires pointer management.
2. Incorrect pointers can cause errors.
3. Each node needs extra pointer memory.
4. Dynamic memory must be released properly.

Conclusion: Self-referential structures are the foundation of linked lists and many dynamic data structures.

8. Apply stack operations PUSH, POP, and DISPLAY using arrays. Develop a C program.

Definition

A stack is a linear data structure that follows the LIFO principle.

LIFO means Last In, First Out. The element inserted last is removed first.

Stack Operations

- PUSH: Insert an element.
- POP: Remove the top element.
- DISPLAY: Display stack elements.
- Overflow: Occurs when attempting to push into a full stack.
- Underflow: Occurs when attempting to pop from an empty stack.

Stack Diagram

Stack Example



30 ← TOP

20

10

PUSH: 10, 20, 30POP removes 30 first.

C Program

```
#include <stdio.h>

#define SIZE 5

int stack[SIZE];
int top = -1;

void push(int value) {
   if (top == SIZE - 1) {
      printf("Stack Overflow\n");
   } else {
      top++;
      stack[top] = value;
      printf("%d pushed\n", value);
   }
}

void pop() {
   if (top == -1) {
      printf("Stack Underflow\n");
   } else {
```



```
        printf("%d popped\n", stack[top]);
        top--;
    }
}

void display() {
    int i;

    if (top == -1) {
        printf("Stack is empty\n");
    } else {
        printf("Stack elements:\n");

        for (i = top; i >= 0; i--) {
            printf("%d\n", stack[i]);
        }
    }
}

int main() {
    push(10);
    push(20);
    push(30);

    display();
```



```
    pop();

    display();

    return 0;
}
```

Output

```
10 pushed
20 pushed
30 pushed
Stack elements:
30
20
10
30 popped
Stack elements:
20
10
```

Step-by-Step Execution

Operation	Stack	TOP
Initial	Empty	-1
PUSH 10	10	0
PUSH 20	10, 20	1
PUSH 30	10, 20, 30	2
POP	10, 20	1



Advantages

1. Simple implementation.
2. PUSH and POP take O(1) time.
3. Useful for expression evaluation.
4. Supports recursion and function calls.

Disadvantages

1. Array stack has a fixed capacity.
2. Overflow occurs when full.
3. Only the top element is directly accessible.

Conclusion: An array-based stack efficiently performs PUSH, POP, and DISPLAY operations.

9. Evaluate the postfix expressions. Assume A = 1, B = 2, C = 3.

Postfix Evaluation Rules

1. Scan the expression from left to right.
2. Push operands onto the stack.
3. When an operator appears, pop two operands.
4. Apply the operator in the correct order.
5. Push the result.
6. The final stack value is the answer.

For exponentiation, $ is treated as the power operator.

(a) A B + C - B A + C $ -

Substitute:

- A = 1
- B = 2
- C = 3

Expression:



1 2 + 3 - 2 1 + 3 $ -

Token	Operation	Stack
1	Push	1
2	Push	1, 2
+	1 + 2 = 3	3
3	Push	3, 3
-	3 - 3 = 0	0
2	Push	0, 2
1	Push	0, 2, 1
+	2 + 1 = 3	0, 3
3	Push	0, 3, 3
$	$3^3 = 27$	0, 27
-	0 - 27 = -27	-27

Final Answer: -27

(b) A B C + * C B A - + *

Substitute:

1 2 3 + * 3 2 1 - + *

Token	Operation	Stack
1	Push	1
2	Push	1, 2
3	Push	1, 2, 3
+	2 + 3 = 5	1, 5
*	1 × 5 = 5	5

3	Push	5, 3
2	Push	5, 3, 2
1	Push	5, 3, 2, 1
-	2 - 1 = 1	5, 3, 1
+	3 + 1 = 4	5, 4
*	5 × 4 = 20	20

Final Answer: 20

Conclusion: Postfix expressions can be evaluated systematically using a stack without parentheses.

10. Convert the following infix expressions to postfix using a stack.

Operator Precedence

Operator	Precedence	Associativity
$ (power)	Highest	Right to left
*, /	Medium	Left to right
+, -	Lowest	Left to right

Parentheses have priority over operators.

(a) A $ B * C - D + E / F / (G + H)

Step-by-step:

1. G + H becomes GH+.
2. E / F becomes EF/.
3. (E/F)/(G+H) becomes EF/GH+/.
4. B*C becomes BC*.
5. A$B becomes AB$.
6. A$B*C becomes AB$C*.
7. Subtract D.

8. Add the final division term.

Postfix:

AB$C*D-EF/GH+/+

(b) A - B / (C * D $ E)

1. D $ E → DE$
2. C * (D $ E) → CDE$*
3. B / (...) → BCDE$*/
4. A - (...) → ABCDE$*/-

Postfix:

ABCDE$*/-

(c) ((A-(B+C))*D)$(E+F)

1. B + C → BC+
2. A - (B+C) → ABC+-
3. Multiply by D → ABC+-D*
4. E + F → EF+
5. Apply power operator.

Postfix:

ABC+-D*EF+$

(d) (a * b) + c / d

1. a * b → ab*
2. c / d → cd/
3. Add the results.

Postfix:

ab*cd/+

(e) (((a / b)-c) + (d * e)) - (a * c)

1. a / b → ab/



2. (a/b)-c → ab/c-
3. d*e → de*
4. Add both expressions → ab/c-de*+
5. a*c → ac*
6. Subtract.

Postfix:

ab/c-de*+ac*-

Conclusion: The stack method preserves operator precedence and associativity during infix-to-postfix conversion.

11. Describe the steps involved in converting infix to postfix using stack operations.

Definition

Infix notation: Operators are written between operands.

Example: A + B

Postfix notation: Operators are written after operands.

Example: AB+

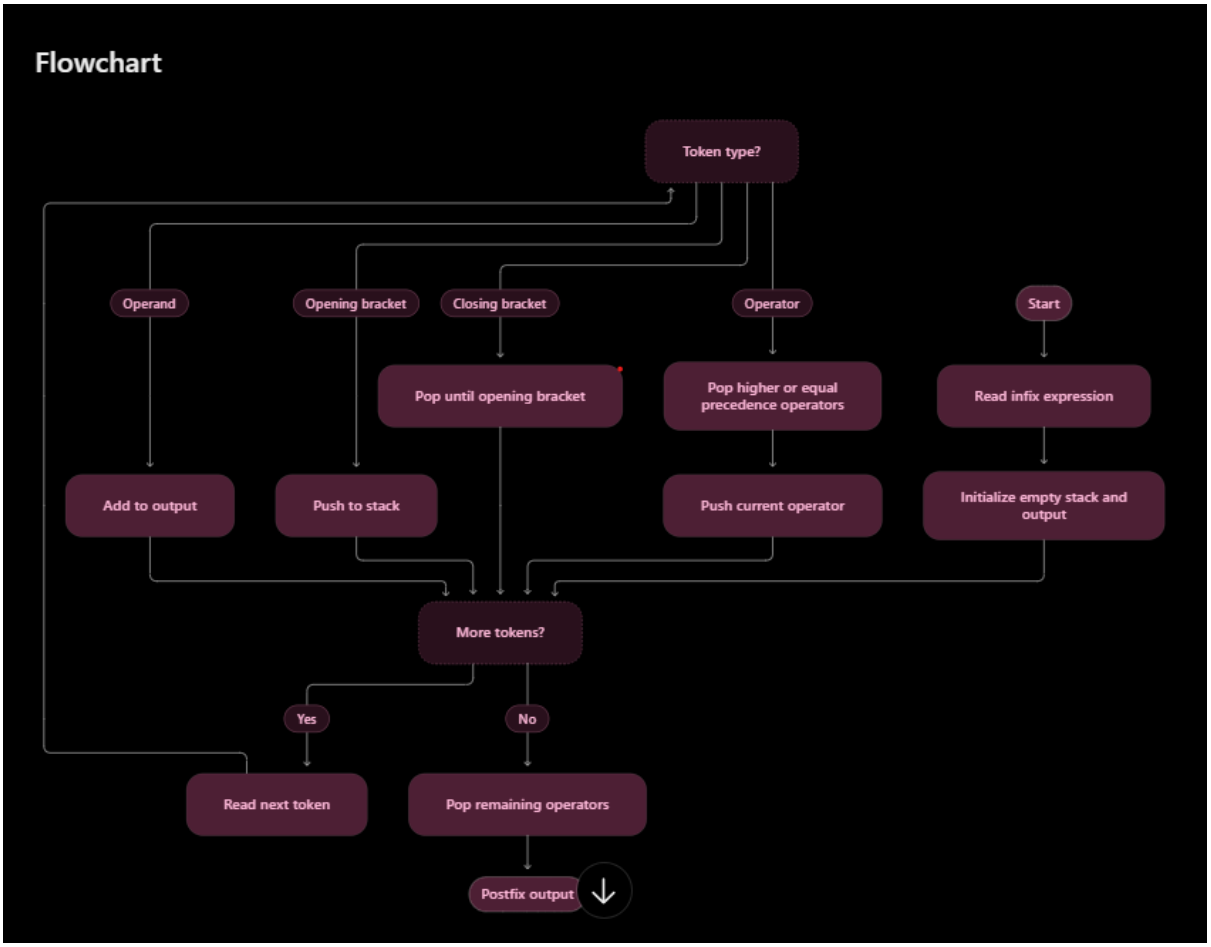
Algorithm

1. Initialize an empty operator stack.
2. Scan the infix expression from left to right.
3. If the token is an operand, add it to the output.
4. If the token is (, push it onto the stack.
5. If the token is), pop operators until (is found. Discard the parentheses.
6. If the token is an operator, pop operators of higher precedence from the stack.
7. For left-associative operators, also pop operators of equal precedence.
8. Push the current operator.
9. After scanning, pop remaining operators.



10. The resulting output is postfix notation.

Flowchart



Example

Convert:

A + B * C

Token	Stack	Output
A	Empty	A
+	+	A
B	+	AB
*	+, *	AB
C	+, *	ABC

End Empty ABC*+

Advantages

1. Removes the need for parentheses in postfix.
2. Supports stack-based evaluation.
3. Respects operator precedence.
4. Simplifies expression processing.

Disadvantages

1. Requires correct precedence rules.
2. More difficult for humans to read.
3. Invalid expressions must be handled separately.

Conclusion: A stack converts infix expressions into postfix form while preserving the intended order of operations.

12. Develop a C function to evaluate postfix expressions using a stack.

Definition

Postfix evaluation is the process of calculating an expression in which operators appear after operands.

Algorithm

1. Create an empty stack.
2. Scan the postfix expression.
3. Push operands.
4. When an operator appears, pop operand B and then operand A.
5. Calculate A operator B.
6. Push the result.
7. At the end, the top of the stack contains the final answer.

C Function

The following function evaluates postfix expressions containing single-digit non-negative operands and the operators +, -, *, and /.



```
#include <stdio.h>
#include <ctype.h>

#define SIZE 100

int evaluatePostfix(char exp[]) {
   int stack[SIZE];
   int top = -1;
   int i, a, b;

   for (i = 0; exp[i] != '\0'; i++) {

      if (isdigit((unsigned char)exp[i])) {
         stack[++top] = exp[i] - '0';
      }

      else {
         b = stack[top--];
         a = stack[top--];

         switch (exp[i]) {
            case '+':
               stack[++top] = a + b;
               break;

            case '-':
```



```
            stack[++top] = a - b;
            break;

        case '*':
            stack[++top] = a * b;
            break;

        case '/':
            if (b == 0) {
                printf("Division by zero");
                return 0;
            }
            stack[++top] = a / b;
            break;
      }
    }
  }

  return stack[top];
}

int main() {
  char exp[] = "23*54*+";

  printf("Result = %d\n", evaluatePostfix(exp));
```



```
    return 0;
}
```

Output:

Result = 26

Demonstration

Expression:

2 3 * 5 4 * +

Token	Operation	Stack
2	Push	2
3	Push	2, 3
*	2 × 3 = 6	6
5	Push	6, 5
4	Push	6, 5, 4
*	5 × 4 = 20	6, 20
+	6 + 20 = 26	26

Advantages

1. Simple evaluation method.
2. No need to process parentheses.
3. Operator precedence is already encoded.
4. Suitable for calculators and compilers.

Disadvantages

1. Requires a stack.
2. Invalid expressions need checking.
3. This example supports only single-digit operands.
4. Division by zero must be handled.



Conclusion: Stack-based postfix evaluation provides a simple and efficient method for evaluating arithmetic expressions.

13. Develop iterative and recursive C programs to calculate factorial.

Definition

The factorial of a non-negative integer n is the product of all positive integers from 1 to n.

It is represented as:

\[n! = n \times (n-1) \times \cdots \times 1 \]

Special case:

\[0! = 1 \]

Example:

\[5! = 5 \times 4 \times 3 \times 2 \times 1 = 120 \]

A. Iterative Factorial

An iterative method uses a loop.

```
#include <stdio.h>


int factorial(int n) {
   int i;
   int fact = 1;


   for (i = 1; i <= n; i++) {
      fact = fact * i;
   }


   return fact;
}
```

```
int main() {
   printf("Factorial = %d\n", factorial(5));
   return 0;
}
```

Output:

Factorial = 120

Working

Iteration	Calculation	Result
Initial	fact = 1	1
1	1 × 1	1
2	1 × 2	2
3	2 × 3	6
4	6 × 4	24
5	24 × 5	120

B. Recursive Factorial

A recursive function calls itself until it reaches a base condition.

```
#include <stdio.h>

int factorial(int n) {
   if (n == 0)
      return 1;

   return n * factorial(n - 1);
}
```



```
int main() {
   printf("Factorial = %d\n", factorial(5));
   return 0;
}
```

Output:

Factorial = 120

Recursive Working

```
factorial(5)
   5 * factorial(4)
      4 * factorial(3)
         3 * factorial(2)
            2 * factorial(1)
               1 * factorial(0)
                  1  ← Base condition
```

Returning:

5 × 4 × 3 × 2 × 1 = 120

Difference

Iterative	**Recursive**
Uses loops.	Uses function calls.
No recursive call stack.	Uses call stack.
Generally less memory overhead.	Requires additional stack memory.
Easy to trace using a loop.	Easy to express mathematically.

Advantages



- Iterative: efficient and avoids recursive stack overhead.
- Recursive: simple and closely follows factorial definition.

Disadvantages

- Iterative: requires loop logic.
- Recursive: can cause stack overflow for large inputs.

Conclusion: Both methods calculate factorial, but iteration generally uses less memory while recursion provides a natural mathematical representation.

14. Discuss Tower of Hanoi with an example using a recursive function.

Definition

The Tower of Hanoi is a mathematical puzzle consisting of three rods and a number of disks of different sizes.

The objective is to move all disks from the source rod to the destination rod using the auxiliary rod.

Rules

1. Only one disk can be moved at a time.
2. Only the top disk of a rod can be moved.
3. A larger disk cannot be placed on a smaller disk.

Diagram

Tower of Hanoi – 3 Disks

Source A

1

2

3

Auxiliary B

Destination C

Move all disks from A to C, using B.

Recursive Algorithm



For n disks:

1. Move n-1 disks from source to auxiliary.
2. Move the largest disk from source to destination.
3. Move n-1 disks from auxiliary to destination.

C Program

```
#include <stdio.h>

void tower(int n, char source,
        char auxiliary, char destination) {

   if (n == 1) {
      printf("Move disk 1 from %c to %c\n",
           source, destination);
      return;
   }

   tower(n - 1, source, destination, auxiliary);

   printf("Move disk %d from %c to %c\n",
        n, source, destination);

   tower(n - 1, auxiliary, source, destination);
}

int main() {
   tower(3, 'A', 'B', 'C');
```



```
    return 0;
}
```

Output

Move disk 1 from A to C

Move disk 2 from A to B

Move disk 1 from C to B

Move disk 3 from A to C

Move disk 1 from B to A

Move disk 2 from B to C

Move disk 1 from A to C

Minimum Number of Moves

$$T(n) = 2T(n-1)+1$$

Therefore:

$$T(n)=2^n-1$$

For three disks:

$$2^3-1=7$$

Advantages

1. Demonstrates recursion clearly.
2. Solves a mathematical puzzle systematically.
3. Helps understand recursive problem decomposition.

Disadvantages

1. Number of moves increases exponentially.
2. Not efficient for large numbers of disks.
3. Recursive calls require stack memory.

Conclusion: Tower of Hanoi is a classic example of recursion and requires a minimum of 2^n-1 moves.

15. Explain Fibonacci sequence using a recursive function.

Definition

The Fibonacci sequence is a sequence in which each number is the sum of the previous two numbers.

The sequence begins:

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

Mathematical Definition

\[F(0)=0 \]

\[F(1)=1 \]

\[F(n)=F(n-1)+F(n-2) \]

Recursive C Program

```
#include <stdio.h>

int fibonacci(int n) {
   if (n == 0)
      return 0;

   if (n == 1)
      return 1;

   return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
   int i;
```

```
  for (i = 0; i < 7; i++) {
    printf("%d ", fibonacci(i));
  }

  return 0;
}
```

Output:

0 1 1 2 3 5 8

Working

For fibonacci(4):

fibonacci(4)

= fibonacci(3) + fibonacci(2)

= 2 + 1

= 3

Recursive Diagram

```
          F(4)
        /     \
      F(3)     F(2)
     /   \     /  \
   F(2) F(1) F(1) F(0)
   /  \
 F(1) F(0)
```

Advantages

1. Simple mathematical implementation.
2. Demonstrates recursive function calls.



3. Useful for understanding recurrence relations.

Disadvantages

1. Recalculates many values.
2. Exponential time complexity for this simple recursive version.
3. Uses recursive stack memory.

Conclusion: Fibonacci recursion calculates each term using the sum of the two previous terms.

16. Describe declaration, initialization, and memory representation of an array in C.

Definition

An array is a collection of elements of the same data type stored in contiguous memory locations.

Each element is accessed using an index.

Declaration

Syntax:

data_type array_name[size];

Example:

int marks[5];

It declares an integer array containing five elements.

Initialization

Example:

int marks[5] = {10, 20, 30, 40, 50};

Elements can be accessed using indexes from 0 to 4.

Memory Representation

Assume:

- Base address = 1000
- Integer size = 4 bytes



Array: marks[5]

10

Index 0

20

Index 1

30

Index 2

40

Index 3

50

Index 4

Each integer occupies 4 bytes in this example.

Address Calculation

For a one-dimensional array:

\[Address(A[i])=Base(A)+i\times Size \]

For marks[3]:

\[1000+3\times4=1012 \]

So the address of marks[3] is 1012.

C Program

```
#include <stdio.h>


int main() {
   int marks[5] = {10, 20, 30, 40, 50};
   int i;


   for (i = 0; i < 5; i++) {
```



```
    printf("marks[%d] = %d\n", i, marks[i]);
  }

  return 0;
}
```

Advantages

1. Stores multiple elements under one name.
2. Direct access using indexes.
3. Easy traversal and sorting.
4. Contiguous memory supports efficient access.

Disadvantages

1. Fixed size in ordinary array declarations.
2. Stores elements of the same type.
3. Insertion and deletion may require shifting.
4. Out-of-bounds access is unsafe.

Conclusion: Arrays provide efficient indexed storage of homogeneous elements in contiguous memory.

17. Analyze pointer declaration, initialization, and dereferencing in C.

Definition

A pointer is a variable that stores the memory address of another variable.

Pointers allow indirect access to data stored in memory.

1. Pointer Declaration

Syntax:

data_type *pointer_name;

Example:

int *p;



Here, p is a pointer to an integer.

2. Pointer Initialization

A pointer can store the address of a variable using the address-of operator &.

int x = 10;

int *p = &x;

Here:

- x stores the value 10.
- &x gives the address of x.
- p stores that address.

3. Dereferencing

The dereference operator * accesses the value stored at the address.

printf("%d", *p);

Output:

10

Pointer Diagram

Pointer Representation

Pointer p

Address: 1000Value: 2000

Variable x

Address: 2000Value: 10

p stores the address of x; *p accesses x's value.

C Program

```
#include <stdio.h>


int main() {
   int x = 10;
```



```
    int *p = &x;

    printf("Value of x = %d\n", x);
    printf("Address of x = %p\n", (void *)&x);
    printf("Value stored in p = %p\n", (void *)p);
    printf("Value pointed by p = %d\n", *p);

    *p = 25;

    printf("Updated x = %d\n", x);

    return 0;
}
```

Output (address varies):

Value of x = 10

Address of x = 0x...

Value stored in p = 0x...

Value pointed by p = 10

Updated x = 25

Pointer Operators

Operator	Meaning	Example
&	Address-of operator	&x
*	Dereference operator	*p

Comparison

Declaration	Initialization	Dereferencing



Creates a pointer variable.	Stores an address in the pointer.	Accesses the value at that address.
int *p;	p = &x;	*p
Does not necessarily point to valid data yet.	Pointer refers to x.	Reads or modifies x.

Advantages

1. Supports dynamic memory allocation.
2. Enables linked lists and trees.
3. Allows functions to modify variables through addresses.
4. Supports efficient array and memory access.

Disadvantages

1. Incorrect pointers may cause crashes.
2. Uninitialized pointers are unsafe.
3. Dangling pointers may refer to released memory.
4. Pointer arithmetic must be used carefully.

Conclusion: Pointers provide indirect memory access and are essential for dynamic data structures in C.

18. Define Algorithm. State the criteria that algorithms must satisfy. Explain with an example.

Definition

An algorithm is a finite sequence of well-defined steps used to solve a particular problem or perform a computation.

An algorithm accepts input, processes it, and produces the required output.

Criteria of an Algorithm

An algorithm should satisfy the following characteristics:

1. Input: It accepts zero or more clearly specified inputs.
2. Output: It produces at least one result.



3. Definiteness: Each step must be clear and unambiguous.
4. Finiteness: It must terminate after a finite number of steps.
5. Effectiveness: Each operation must be simple and executable.
6. Correctness: It must produce the correct result for valid inputs.
7. Generality: It should solve all valid instances of the intended problem.

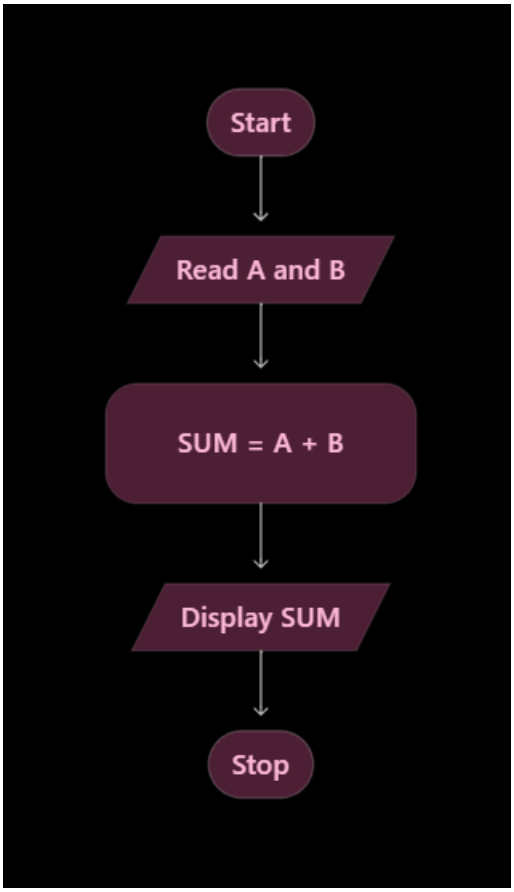
Example: Algorithm to Add Two Numbers

Problem: Add two numbers and display their sum.

Algorithm

1. Start.
2. Read two numbers A and B.
3. Calculate SUM = A + B.
4. Display SUM.
5. Stop.

Flowchart



C Program

```
#include <stdio.h>

int main() {
   int a, b, sum;

   printf("Enter two numbers: ");
   scanf("%d %d", &a, &b);

   sum = a + b;

   printf("Sum = %d\n", sum);
```

```
    return 0;
}
```

Example:

Input:

10 20

Output:

Sum = 30

Explanation of Criteria

Criterion	Example
Input	Two numbers are entered.
Output	Their sum is displayed.
Definiteness	Addition is clearly specified.
Finiteness	The program ends after displaying the sum.
Effectiveness	Addition and printing are executable operations.
Correctness	10 + 20 gives 30.
Generality	It works for other valid pairs of integers.

Advantages

1. Provides a systematic problem-solving approach.
2. Makes programs easier to understand.
3. Helps identify errors before coding.
4. Supports analysis of time and memory requirements.

Disadvantages

1. Complex problems may require lengthy algorithms.
2. Multiple algorithms may exist for the same problem.



3. An algorithm does not itself specify all programming-language details.

MODULE 2

1. Apply ENQUEUE and DEQUEUE operations of a linear queue with a suitable example.

Definition

A queue is a linear data structure that follows the FIFO (First In, First Out) principle.

The element inserted first is removed first.

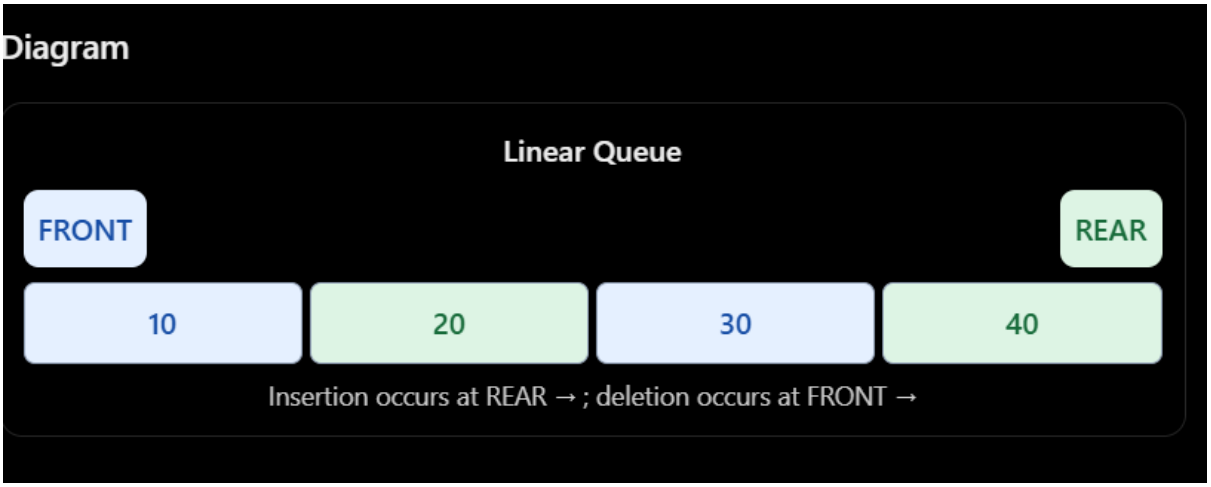
A queue has two ends:

- FRONT: Points to the first element; deletion takes place here.
- REAR: Points to the last element; insertion takes place here.

Basic Operations

1. ENQUEUE: Inserts an element at the rear.
2. DEQUEUE: Removes an element from the front.
3. PEEK: Returns the front element without removing it.
4. DISPLAY: Displays all queue elements.
5. Overflow: Occurs when insertion is attempted into a full queue.
6. Underflow: Occurs when deletion is attempted from an empty queue.

Diagram



Step-by-Step Example

Initially:

Queue = Empty

FRONT = -1

REAR = -1

ENQUEUE(10)

Insert 10 into the queue.

[10] [ ] [ ] [ ]

F,R

FRONT = 0, REAR = 0

ENQUEUE(20)

[10] [20] [ ] [ ]

F R

FRONT = 0, REAR = 1

ENQUEUE(30)

[10] [20] [30] [ ]

F R

FRONT = 0, REAR = 2

DEQUEUE()

Remove the front element, 10.

[ ] [20] [30] [ ]

F R

FRONT = 1, REAR = 2

ENQUEUE(40)

[ ] [20] [30] [40]

F R



FRONT = 1, REAR = 3

Algorithms

ENQUEUE

ENQUEUE(Q, ITEM)

1. If REAR = SIZE - 1
   Print "Overflow"
2. Else
   If FRONT = -1
      FRONT = 0
3. REAR = REAR + 1
4. Q[REAR] = ITEM
5. Stop

DEQUEUE

DEQUEUE(Q)

1. If FRONT = -1
   Print "Underflow"
2. Else
   ITEM = Q[FRONT]
3. If FRONT = REAR
   FRONT = REAR = -1
4. Else
   FRONT = FRONT + 1
5. Return ITEM

Advantages



1. Simple implementation.
2. ENQUEUE and DEQUEUE are O(1) in a basic array queue.
3. Maintains FIFO order.
4. Useful for scheduling and buffering.

Disadvantages

1. Fixed-size array may become full.
2. Deleted spaces at the beginning are not reused in a simple linear queue.
3. May cause false overflow.

Conclusion: A linear queue inserts at the rear and deletes from the front, following FIFO.

2. Implement a circular queue in C and demonstrate how it solves the space-utilization problem.

Definition

A circular queue is a queue in which the last array position is logically connected to the first position.

It uses the available array space again by wrapping the rear index to the beginning.

Why Circular Queue?

In an ordinary linear queue, when REAR reaches the last array position, insertion may fail even when earlier positions are empty.

A circular queue solves this problem by reusing those vacant positions.

Diagram

Circular Queue – Wrap Around

50

Index 0

60

Index 1

30



Index 2

40

Index 3

After index 3, next position is index 0.

Conditions

For a circular queue with SIZE positions:

- Empty: count == 0
- Full: count == SIZE
- Next rear position: (front + count) % SIZE
- Next front position after deletion: (front + 1) % SIZE

Using a count variable makes full and empty conditions unambiguous.

C Program

```
#include <stdio.h>

#define SIZE 5

int queue[SIZE];
int front = 0;
int count = 0;

void enqueue(int value) {
   int rear;

   if (count == SIZE) {
      printf("Queue Overflow\n");
      return;
```



```
    }

    rear = (front + count) % SIZE;
    queue[rear] = value;
    count++;

    printf("%d inserted\n", value);
}

void dequeue() {
    if (count == 0) {
        printf("Queue Underflow\n");
        return;
    }

    printf("%d deleted\n", queue[front]);

    front = (front + 1) % SIZE;
    count--;
}

void display() {
    int i;

    if (count == 0) {
        printf("Queue is empty\n");
```



```
        return;
    }

    printf("Queue: ");

    for (i = 0; i < count; i++) {
        printf("%d ", queue[(front + i) % SIZE]);
    }

    printf("\n");
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    enqueue(40);
    enqueue(50);

    display();

    dequeue();
    dequeue();

    enqueue(60);
    enqueue(70);
```



```
    display();

    return 0;
}
```

Output

10 inserted

20 inserted

30 inserted

40 inserted

50 inserted

Queue: 10 20 30 40 50

10 deleted

20 deleted

60 inserted

70 inserted

Queue: 30 40 50 60 70

Execution

Operation	Queue (logical order)
Enqueue 10, 20, 30, 40, 50	10, 20, 30, 40, 50
Dequeue	20, 30, 40, 50
Dequeue	30, 40, 50
Enqueue 60	30, 40, 50, 60
Enqueue 70	30, 40, 50, 60, 70

The last two elements use the freed array positions by wrapping around.



Advantages

1. Reuses vacant positions.
2. Avoids shifting elements.
3. Efficient ENQUEUE and DEQUEUE.
4. Better space utilization than a simple linear array queue.

Disadvantages

1. Index calculation is more complex.
2. Full and empty conditions must be handled correctly.
3. Fixed-size circular arrays still have limited capacity.

Conclusion: A circular queue improves array space utilization by wrapping around and reusing previously freed positions.

3. Develop a C program that performs the basic operations of a circular queue.

This question is similar to Question 2, but focuses on a complete implementation of circular queue operations.

Definition

A circular queue is a FIFO data structure where the last position connects logically to the first position.

Operations

- Enqueue
- Dequeue
- Display
- Check empty
- Check full

C Program

```
#include <stdio.h>


#define SIZE 5
```



```
int q[SIZE];
int front = 0;
int count = 0;

void enqueue(int x) {
   int rear;

   if (count == SIZE) {
      printf("Overflow\n");
      return;
   }

   rear = (front + count) % SIZE;
   q[rear] = x;
   count++;
}

void dequeue() {
   if (count == 0) {
      printf("Underflow\n");
      return;
   }

   printf("Deleted: %d\n", q[front]);
```



```
    front = (front + 1) % SIZE;
    count--;
}

void display() {
    int i;

    if (count == 0) {
        printf("Empty queue\n");
        return;
    }

    for (i = 0; i < count; i++) {
        printf("%d ", q[(front + i) % SIZE]);
    }

    printf("\n");
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);

    display();
```



```
    dequeue();

    enqueue(40);
    enqueue(50);

    display();

    return 0;
}
```

Output

10 20 30

Deleted: 10

20 30 40 50

Advantages

1. Efficient memory reuse.
2. No element shifting.
3. Simple FIFO operation.
4. Enqueue and dequeue take O(1) time.

Disadvantages

1. Capacity is fixed.
2. Wrap-around logic must be correct.
3. Full queue cannot accept additional elements.

Conclusion: Circular queues provide efficient FIFO operations and reuse vacant positions in an array.

4. Analyze insertion and deletion operations of a deque at both FRONT and REAR.

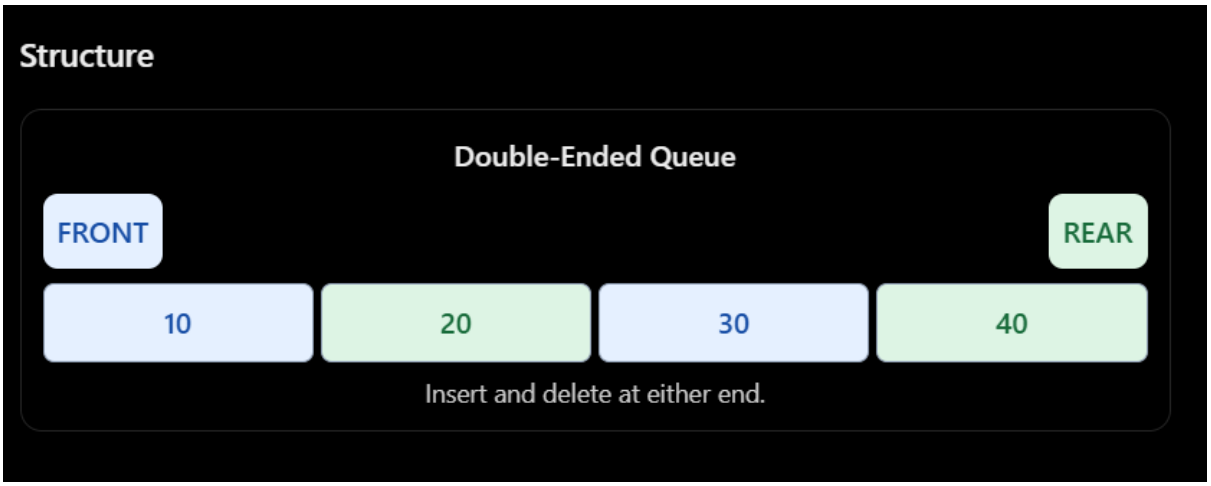
Definition



A deque (Double-Ended Queue) is a linear data structure in which insertion and deletion can be performed at both ends: FRONT and REAR.

Deque means Double-Ended Queue.

Structure



Operations

Operation	Description
Insert Front	Adds an element at the front.
Insert Rear	Adds an element at the rear.
Delete Front	Removes the front element.
Delete Rear	Removes the rear element.

Step-by-Step Example

Initially:

Deque = Empty

Insert Rear(10)

FRONT → [10] ← REAR

Insert Rear(20)

FRONT → [10] [20] ← REAR

Insert Front(5)

FRONT → [5] [10] [20] ← REAR

Insert Rear(30)

FRONT → [5] [10] [20] [30] ← REAR

Delete Front()

Removes 5.

FRONT → [10] [20] [30] ← REAR

Delete Rear()

Removes 30.

FRONT → [10] [20] ← REAR

Algorithms

Insert Front

1. Check whether deque is full.
2. If empty:

    FRONT = REAR = 0
3. Else:

    FRONT = (FRONT - 1 + SIZE) % SIZE
4. DEQUE[FRONT] = ITEM
5. Stop

Insert Rear

1. Check whether deque is full.
2. If empty:

    FRONT = REAR = 0
3. Else:



REAR = (REAR + 1) % SIZE

4. DEQUE[REAR] = ITEM

5. Stop

Delete Front

1. Check whether deque is empty.

2. ITEM = DEQUE[FRONT]

3. If FRONT = REAR:

FRONT = REAR = -1

4. Else:

FRONT = (FRONT + 1) % SIZE

5. Return ITEM

Delete Rear

1. Check whether deque is empty.

2. ITEM = DEQUE[REAR]

3. If FRONT = REAR:

FRONT = REAR = -1

4. Else:

REAR = (REAR - 1 + SIZE) % SIZE

5. Return ITEM

Advantages

1. Insertion and deletion at both ends.
2. Flexible queue operations.
3. Useful in scheduling and sliding-window problems.
4. Circular-array implementation can perform end operations in O(1).

Disadvantages

1. More complex than an ordinary queue.



2. Requires careful front/rear management.
3. Fixed array implementation has limited capacity.

Conclusion: A deque supports insertion and deletion at both FRONT and REAR.

5. Explain different types of Deques, including input-restricted and output-restricted Deques.

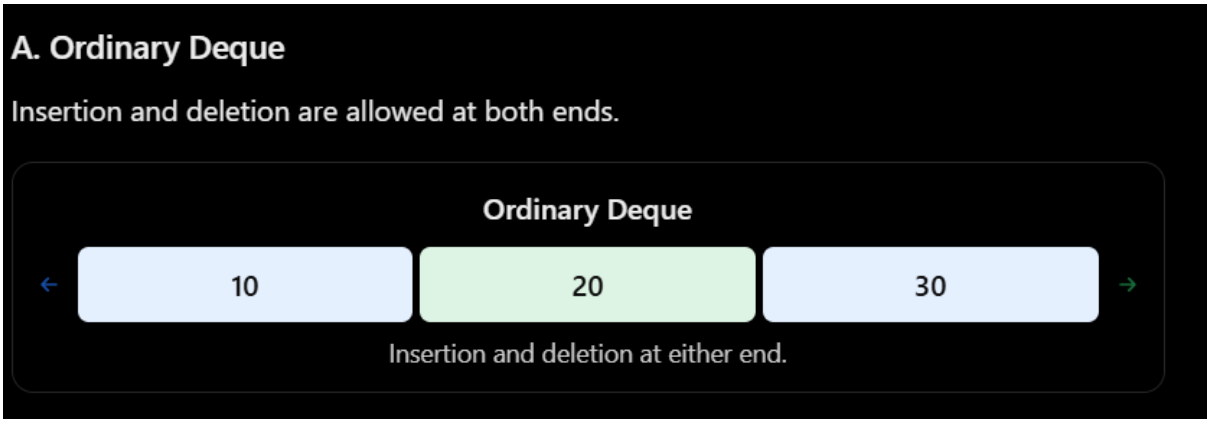
Definition

A deque is a double-ended queue that allows insertion and deletion at both ends.

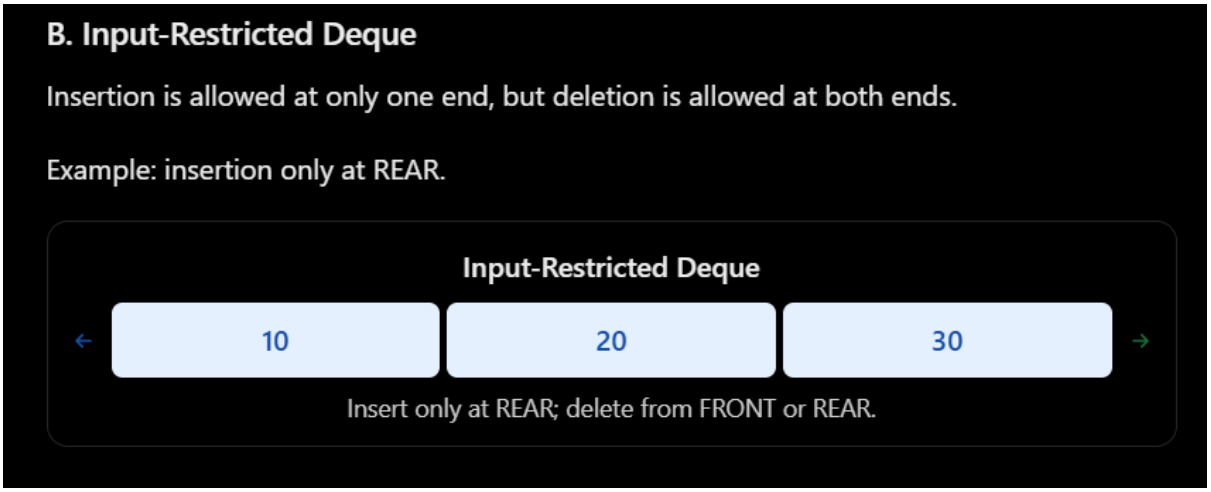
There are three commonly discussed types:

1. Ordinary deque
2. Input-restricted deque
3. Output-restricted deque

A. Ordinary Deque



B. Input-Restricted Deque



Example

Start:

[10] [20]

Insert rear 30:

[10] [20] [30]

Delete front:

[20] [30]

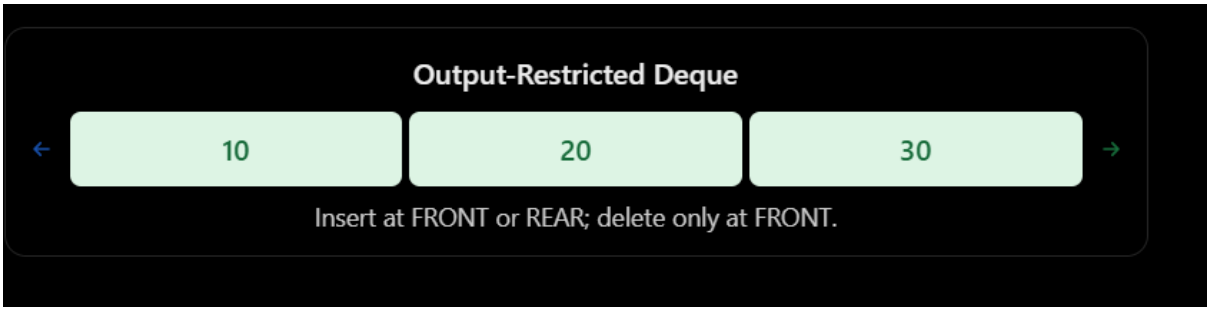
Delete rear:

[20]

C. Output-Restricted Deque

Deletion is allowed at only one end, but insertion is allowed at both ends.

Example: deletion only at FRONT.



Output-Restricted Deque

10

20

30

Insert at FRONT or REAR; delete only at FRONT.

Example

Start:

[10] [20]

Insert front 5:

[5] [10] [20]

Insert rear 30:

[5] [10] [20] [30]

Delete front:

[10] [20] [30]

Comparison

Feature	Ordinary Deque	Input-Restricted	Output-Restricted
Insert Front	Yes	No	Yes
Insert Rear	Yes	Yes	Yes
Delete Front	Yes	Yes	Yes
Delete Rear	Yes	Yes	No

Advantages

1. Flexible insertion and deletion.
2. Supports different processing requirements.
3. Useful in scheduling, buffering, and algorithms.

Disadvantages

1. More complex than a normal queue.



2. Requires careful management of both ends.
3. Restricted versions provide fewer operations.

Conclusion: Deques provide double-ended processing, while restricted deques limit insertion or deletion to one end.

6. Construct a priority queue by assigning priorities and illustrate its representation.

Definition

A priority queue is a data structure in which each element is associated with a priority.

Elements are removed according to priority rather than simply according to insertion order.

For this example, assume smaller priority number means higher priority.

Example

Consider these elements:

Element	Priority
A	3
B	1
C	4
D	2

Priority order:

B → D → A → C

Because priority 1 is highest, B is removed first.

Diagram

Priority Queue

B

P = 1

D

P = 2

A

P = 3

C

P = 4

Front = highest priority; removal order B, D, A, C.

Step-by-Step Construction

1. Insert A with priority 3.
2. Insert B with priority 1.
3. Insert C with priority 4.
4. Insert D with priority 2.
5. Arrange elements according to priority.
6. Remove B first, then D, A, and C.

Representation

A priority queue can be implemented using:

- Arrays
- Linked lists
- Binary heaps

A simple sorted-array representation:

Element: B D A C

Priority: 1 2 3 4

FRONT → B

Advantages

1. Processes important elements first.
2. Useful for CPU scheduling.
3. Useful in shortest-path algorithms.

4. Supports priority-based event handling.

Disadvantages

1. Maintaining priority order may require extra work.
2. Insertion or deletion cost depends on implementation.
3. Equal priorities need a defined ordering policy.

Conclusion: A priority queue removes elements according to priority, not merely arrival time.

7. Classify different types of linked lists and illustrate their structural organization.

Definition

A linked list is a dynamic linear data structure consisting of nodes connected through pointers.

Each node contains data and one or more links to other nodes.

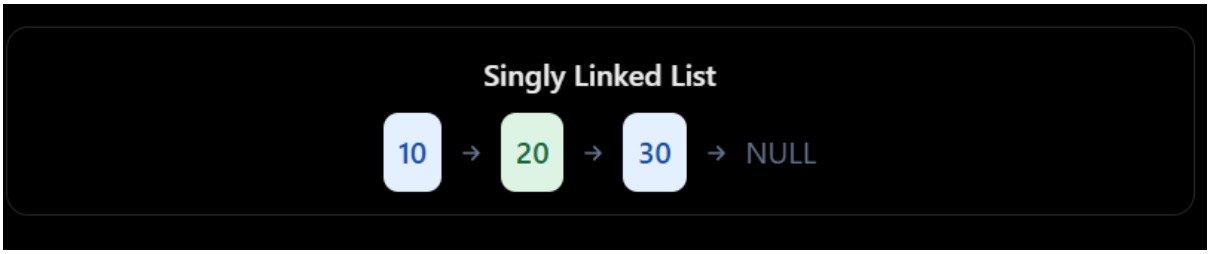
Types of Linked Lists

1. Singly linked list
2. Doubly linked list
3. Circular singly linked list
4. Circular doubly linked list

A. Singly Linked List

Each node contains data and a pointer to the next node.

Singly Linked List



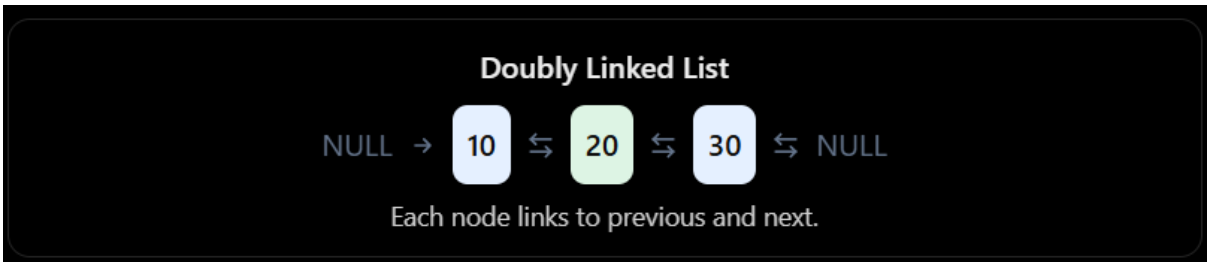
- Traversal is forward only.
- Last node points to NULL.

B. Doubly Linked List

Each node contains:

- Previous pointer
- Data
- Next pointer

Doubly Linked List



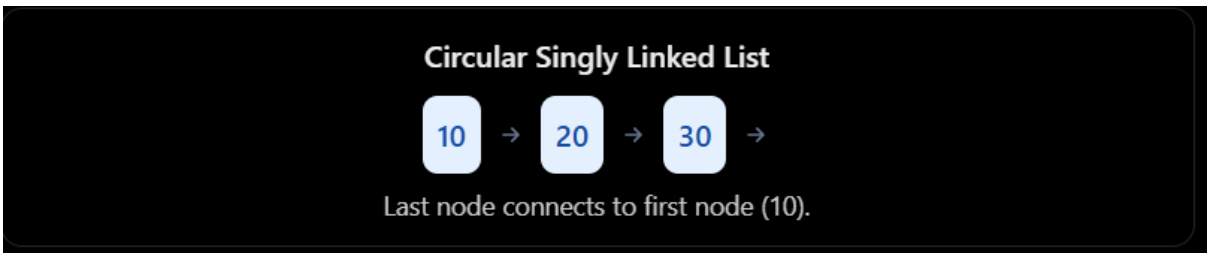
Each node links to previous and next.

- Traversal in both directions.
- Requires extra pointer memory.

C. Circular Singly Linked List

The last node points back to the first node.

Circular Singly Linked List

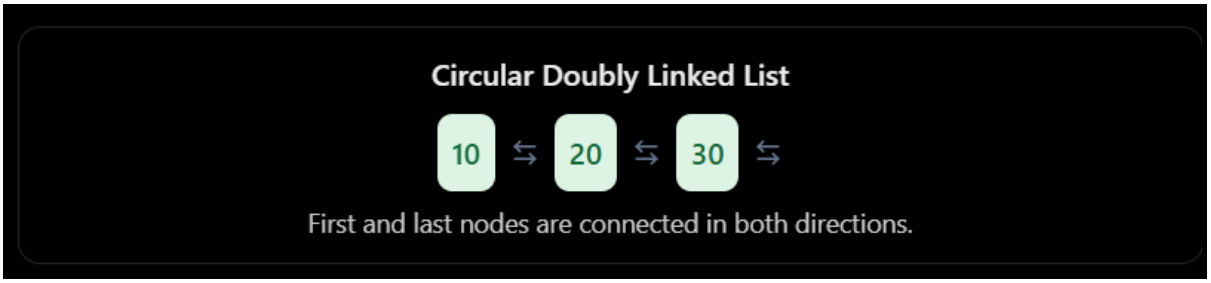


- No NULL at the end.
- Traversal must stop when the starting node is reached again.

D. Circular Doubly Linked List

Each node has previous and next pointers, and the first and last nodes are connected.

Circular Doubly Linked List



Comparison

Type	Pointers per node	Direction	Last node
Singly	One	Forward	NULL
Doubly	Two	Forward and backward	NULL
Circular singly	One	Forward	Points to first
Circular doubly	Two	Both directions	Connects to first

Advantages

1. Dynamic size.
2. Efficient insertion and deletion when position is known.
3. Does not require contiguous memory.
4. Useful for stacks, queues, and other structures.

Disadvantages

1. Extra memory for pointers.
2. No direct index access.
3. Traversal can take O(n).
4. Pointer mistakes may cause errors.

Conclusion: Linked lists organize data using connected nodes and support several forms of sequential and circular traversal.

8. Implement a circular queue using a dynamic array and demonstrate ENQUEUE and DEQUEUE.

Definition

A dynamic circular queue is a circular queue whose array capacity can grow when additional elements are required.

It combines:

- Circular indexing
- Dynamic memory allocation
- FIFO operations

Unlike a fixed circular queue, it can increase capacity when full.

Structure

Circular array:

[10] [20] [30] [40]

When full:

Allocate a larger array

Copy elements in logical order

Reset front and rear positions

C Program

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
   int *data;
   int capacity;
   int front;
   int count;
} Queue;
```

```
void initQueue(Queue *q, int capacity) {
   q->data = malloc(capacity * sizeof(int));
   q->capacity = capacity;
   q->front = 0;
   q->count = 0;
}

int growQueue(Queue *q) {
   int newCapacity = q->capacity * 2;
   int *newData = malloc(newCapacity * sizeof(int));
   int i;

   if (newData == NULL)
      return 0;

   for (i = 0; i < q->count; i++) {
      newData[i] =
         q->data[(q->front + i) % q->capacity];
   }

   free(q->data);

   q->data = newData;
   q->capacity = newCapacity;
   q->front = 0;
```



```
    return 1;
}

void enqueue(Queue *q, int value) {
    int rear;

    if (q->count == q->capacity) {
        if (!growQueue(q)) {
            printf("Memory allocation failed\n");
            return;
        }
    }

    rear = (q->front + q->count) % q->capacity;
    q->data[rear] = value;
    q->count++;
}

void dequeue(Queue *q) {
    if (q->count == 0) {
        printf("Queue is empty\n");
        return;
    }

    printf("Deleted: %d\n", q->data[q->front]);
```



```
    q->front = (q->front + 1) % q->capacity;
    q->count--;
}

void display(Queue *q) {
    int i;

    for (i = 0; i < q->count; i++) {
        printf("%d ",
            q->data[(q->front + i) % q->capacity]);
    }

    printf("\n");
}

int main() {
    Queue q;

    initQueue(&q, 3);

    enqueue(&q, 10);
    enqueue(&q, 20);
    enqueue(&q, 30);

    display(&q);
```



```
    dequeue(&q);

    enqueue(&q, 40);
    enqueue(&q, 50);

    display(&q);

    free(q.data);

    return 0;
}
```

Output

10 20 30

Deleted: 10

20 30 40 50

The queue starts with capacity 3. When the fourth element is inserted, the array grows to capacity 6.

Advantages

1. Capacity can increase dynamically.
2. Reuses positions through circular indexing.
3. Avoids fixed-size overflow when memory is available.
4. No shifting during ordinary enqueue/dequeue.

Disadvantages

1. Resizing requires copying elements.
2. Uses dynamic memory.



3. Allocation may fail.
4. Growing occasionally takes O(n) time.

Conclusion: A dynamic circular queue combines circular space reuse with the ability to grow its capacity.

9. Implement C functions to traverse a singly linked list and search for a specified element.

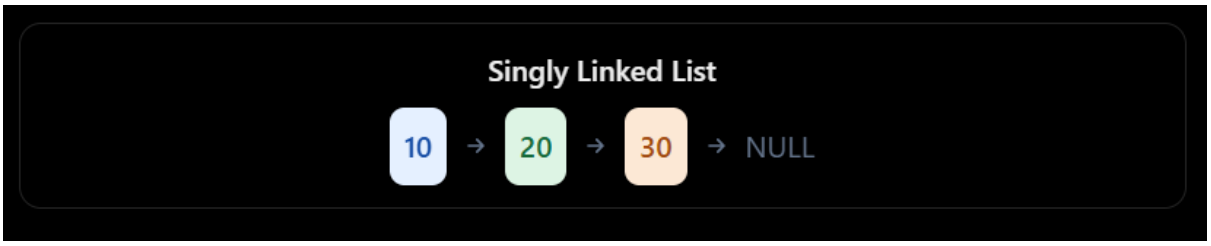
Definition

A singly linked list is a linear data structure in which each node contains data and a pointer to the next node.

Each node is connected to the next node, and the last node points to NULL.

Structure

Singly Linked List



A. Traversal

Traversal means visiting every node in the linked list, starting from the first node.

Algorithm

TRAVERSE(HEAD)

1. PTR = HEAD
2. While PTR != NULL

   Display PTR->DATA

   PTR = PTR->NEXT
3. Stop

B. Searching

Searching means finding whether a specified element exists in the linked list.

Algorithm

SEARCH(HEAD, KEY)

1. PTR = HEAD
2. While PTR != NULL
   If PTR->DATA == KEY
      Return position
   PTR = PTR->NEXT
3. Return NOT FOUND

C Program

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *next;
};

void traverse(struct Node *head) {
   struct Node *ptr = head;

   printf("List: ");

   while (ptr != NULL) {
```



```
        printf("%d ", ptr->data);
        ptr = ptr->next;
    }

    printf("\n");
}

int search(struct Node *head, int key) {
    struct Node *ptr = head;
    int position = 1;

    while (ptr != NULL) {
        if (ptr->data == key)
            return position;

        ptr = ptr->next;
        position++;
    }

    return -1;
}

int main() {
    struct Node *head, *second, *third;
    int result;
```



```
head = malloc(sizeof(struct Node));
second = malloc(sizeof(struct Node));
third = malloc(sizeof(struct Node));

if (head == NULL || second == NULL || third == NULL)
   return 1;

head->data = 10;
head->next = second;

second->data = 20;
second->next = third;

third->data = 30;
third->next = NULL;

traverse(head);

result = search(head, 20);

if (result != -1)
   printf("Found at position %d\n", result);
else
   printf("Not found\n");

free(third);
```



```
    free(second);
    free(head);

    return 0;
}
```

Output

List: 10 20 30

Found at position 2

Step-by-Step Execution

Traversal:

HEAD → 10 → 20 → 30 → NULL

Searching for 20:

Step	Node	Comparison
1	10	Not equal
2	20	Found
3	Stop	Position 2

Advantages

1. Traversal visits all elements.
2. Searching does not require contiguous memory.
3. Linked lists can grow dynamically.

Disadvantages

1. Searching takes O(n) time.
2. No direct index access.
3. Pointer operations require care.

Conclusion: Traversal visits each node, while searching compares node values until the key is found or the list ends.



10. Explain insertion operations in a singly linked list at the beginning, end, and after a specified node.

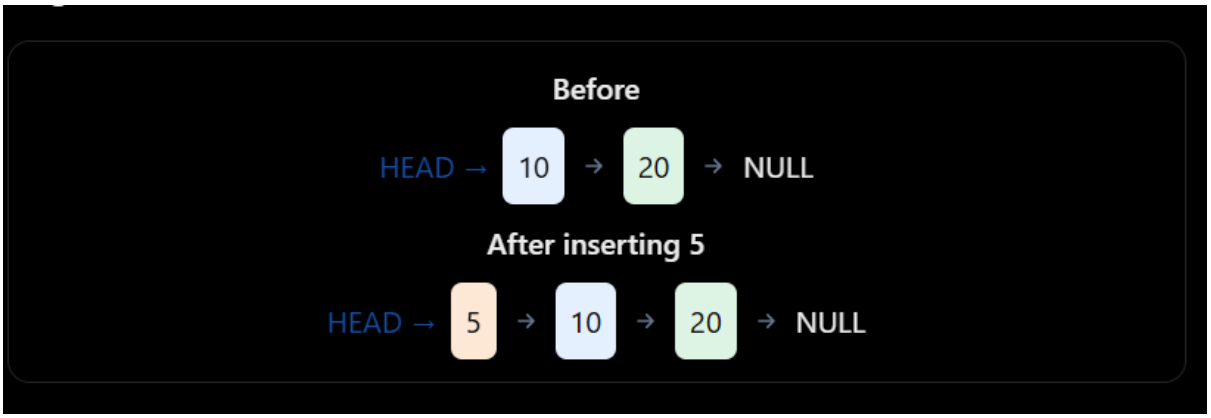
Definition

Insertion is the operation of adding a new node to a linked list.

A new node contains data and a pointer to the next node.

A. Insertion at the Beginning

Diagram



Algorithm

1. Create NEW node.

2. NEW->DATA = ITEM

3. NEW->NEXT = HEAD

4. HEAD = NEW

5. Stop

B. Insertion at the End

Diagram

Before:

HEAD → 10 → 20 → NULL

After inserting 30:

HEAD → 10 → 20 → 30 → NULL

Algorithm

1. Create NEW node.
2. NEW->DATA = ITEM
3. NEW->NEXT = NULL
4. If HEAD = NULL
   HEAD = NEW
5. Else
   PTR = HEAD
   While PTR->NEXT != NULL
      PTR = PTR->NEXT
   PTR->NEXT = NEW
6. Stop

C. Insertion After a Specified Node

Diagram

Before:

10 → 20 → 40 → NULL

Insert 30 after 20:

10 → 20 → 30 → 40 → NULL

Algorithm

1. Find the specified node PTR.
2. If PTR = NULL
   Print "Node not found"
3. Create NEW node.
4. NEW->DATA = ITEM



5. NEW->NEXT = PTR->NEXT

6. PTR->NEXT = NEW

7. Stop

C Program

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *next;
};

struct Node *insertBeginning(struct Node *head, int x) {
   struct Node *newNode = malloc(sizeof(struct Node));

   if (newNode == NULL)
      return head;

   newNode->data = x;
   newNode->next = head;

   return newNode;
}

struct Node *insertEnd(struct Node *head, int x) {
   struct Node *newNode = malloc(sizeof(struct Node));
```



```
    struct Node *ptr;

    if (newNode == NULL)
        return head;

    newNode->data = x;
    newNode->next = NULL;

    if (head == NULL)
        return newNode;

    ptr = head;

    while (ptr->next != NULL)
        ptr = ptr->next;

    ptr->next = newNode;

    return head;
}

void insertAfter(struct Node *ptr, int x) {
    struct Node *newNode;

    if (ptr == NULL)
        return;
```



```
  newNode = malloc(sizeof(struct Node));

  if (newNode == NULL)
    return;

  newNode->data = x;
  newNode->next = ptr->next;
  ptr->next = newNode;
}
```

Advantages

1. Beginning insertion takes O(1).
2. Insertion after a known node takes O(1).
3. No shifting of elements.
4. List can grow dynamically.

Disadvantages

1. End insertion takes O(n) without a tail pointer.
2. Requires memory allocation.
3. Incorrect pointer updates can lose nodes.

Conclusion: Linked-list insertion creates a new node and updates links according to its position.

11. Explain deletion operations in a singly linked list from the beginning, end, and a specified node.

Definition

Deletion is the operation of removing a node from a linked list and adjusting the links so that the remaining nodes stay connected.

A. Deletion from the Beginning



Before:

HEAD → 10 → 20 → 30 → NULL

Delete 10:

HEAD → 20 → 30 → NULL

Algorithm

1. If HEAD = NULL

   Print "List empty"

2. TEMP = HEAD

3. HEAD = HEAD->NEXT

4. Free TEMP

5. Stop

B. Deletion from the End

Before:

HEAD → 10 → 20 → 30 → NULL

Delete 30:

HEAD → 10 → 20 → NULL

Algorithm

1. If HEAD = NULL

   Print "List empty"

2. If HEAD->NEXT = NULL

   Free HEAD

   HEAD = NULL

3. Else

   PTR = HEAD

   While PTR->NEXT->NEXT != NULL

     PTR = PTR->NEXT



TEMP = PTR->NEXT

PTR->NEXT = NULL

Free TEMP

4. Stop

C. Deletion of a Specified Node

Before:

10 → 20 → 30 → 40 → NULL

Delete 30:

10 → 20 → 40 → NULL

Algorithm

1. If HEAD = NULL

Print "List empty"

2. If HEAD->DATA = KEY

TEMP = HEAD

HEAD = HEAD->NEXT

Free TEMP

3. Else

PTR = HEAD

While PTR->NEXT != NULL

If PTR->NEXT->DATA = KEY

TEMP = PTR->NEXT

PTR->NEXT = TEMP->NEXT

Free TEMP

Stop

PTR = PTR->NEXT

4. If not found



Print "Node not found"

Advantages

1. Beginning deletion takes O(1).
2. No shifting of remaining elements.
3. Dynamic memory can be released.
4. Deletion after a known previous node is efficient.

Disadvantages

1. End deletion takes O(n) without a previous pointer.
2. Searching for a value takes O(n).
3. Pointer mistakes may cause memory errors.

Conclusion: Deletion removes a node and reconnects its predecessor and successor appropriately.

12. Develop a C program for creation, traversal, searching, insertion, and deletion in a singly linked list.

Definition

A singly linked list is a dynamic collection of nodes, where each node stores data and a pointer to the next node.

Structure

HEAD → [10] → [20] → [30] → NULL

C Program

This program includes:

- Creation
- Traversal
- Searching
- Insertion at the beginning
- Deletion by value

#include <stdio.h>

```
#include <stdlib.h>

struct Node {
   int data;
   struct Node *next;
};

struct Node *createNode(int value) {
   struct Node *newNode =
      malloc(sizeof(struct Node));

   if (newNode == NULL)
      return NULL;

   newNode->data = value;
   newNode->next = NULL;

   return newNode;
}

void traverse(struct Node *head) {
   struct Node *ptr = head;

   printf("List: ");

   while (ptr != NULL) {
```



```
        printf("%d ", ptr->data);
        ptr = ptr->next;
    }

    printf("\n");
}

int search(struct Node *head, int key) {
    int position = 1;

    while (head != NULL) {
        if (head->data == key)
            return position;

        head = head->next;
        position++;
    }

    return -1;
}

struct Node *insertBeginning(struct Node *head, int x) {
    struct Node *newNode = createNode(x);

    if (newNode == NULL)
        return head;
```



```
    newNode->next = head;

    return newNode;
}

struct Node *deleteValue(struct Node *head, int key) {
    struct Node *ptr = head;
    struct Node *prev = NULL;

    while (ptr != NULL) {
        if (ptr->data == key) {
            if (prev == NULL)
                head = ptr->next;
            else
                prev->next = ptr->next;

            free(ptr);
            return head;
        }

        prev = ptr;
        ptr = ptr->next;
    }

    printf("Element not found\n");
```



```
    return head;
}

void freeList(struct Node *head) {
    struct Node *temp;

    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}

int main() {
    struct Node *head = NULL;
    int result;

    head = insertBeginning(head, 30);
    head = insertBeginning(head, 20);
    head = insertBeginning(head, 10);

    traverse(head);

    result = search(head, 20);
```



```
    if (result != -1)
        printf("Found at position %d\n", result);
    else
        printf("Not found\n");

    head = insertBeginning(head, 5);

    traverse(head);

    head = deleteValue(head, 20);

    traverse(head);

    freeList(head);

    return 0;
}
```

Output

List: 10 20 30

Found at position 2

List: 5 10 20 30

List: 5 10 30

Step-by-Step Execution

Operation	Result
Create 10, 20, 30	10 → 20 → 30



Traverse	Displays 10, 20, 30
Search 20	Found at position 2
Insert 5	5 → 10 → 20 → 30
Delete 20	5 → 10 → 30

Advantages

1. Supports dynamic memory allocation.
2. Insertion and deletion do not require shifting.
3. Functions make the program modular.
4. Supports searching and traversal.

Disadvantages

1. Searching takes O(n).
2. Extra memory is needed for pointers.
3. Dynamic allocation must be managed carefully.

Conclusion: A singly linked list supports dynamic creation and manipulation using pointer-based operations.

13. Analyze the difference between array-based queue and linked-list-based queue.

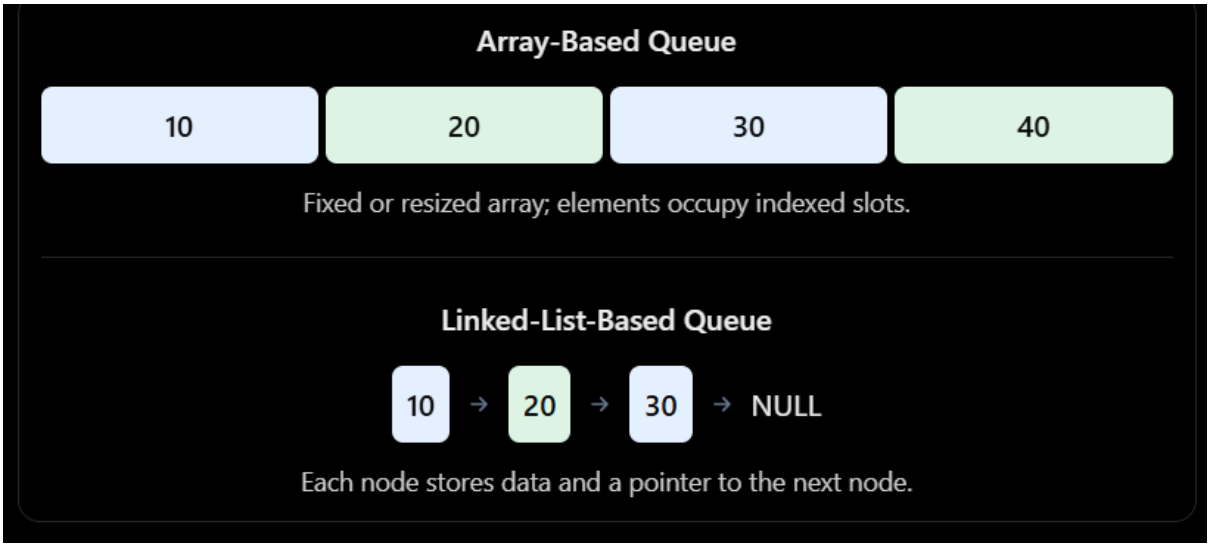
Definition

An array-based queue stores queue elements in an array.

A linked-list-based queue stores queue elements in dynamically allocated nodes connected by pointers.

Both follow FIFO.

Diagram



Comparison

No.	Array-Based Queue	Linked-List-Based Queue
1	Uses an array.	Uses linked nodes.
2	Usually fixed capacity unless resized.	Capacity grows dynamically as memory permits.
3	Elements occupy contiguous array slots.	Nodes may be scattered in memory.
4	No pointer needed per element.	Each node stores a next pointer.
5	Enqueue can be O(1) with circular indexing.	Enqueue is O(1) with a rear pointer.
6	Dequeue can be O(1) with circular indexing.	Dequeue is O(1) with a front pointer.
7	Fixed array overflow occurs when full.	Overflow occurs when memory allocation fails.

8	May require resizing for larger capacity.	Allocates memory for each new node.
9	Memory allocation is simple.	Requires dynamic allocation and freeing.
10	Often has better locality of reference.	Pointer traversal may have poorer locality.

Memory Allocation

Array Queue

int queue[100];

Memory for 100 integers is reserved.

Linked Queue

```
struct Node {
   int data;
   struct Node *next;
};
```

Each new element is allocated separately using malloc().

Advantages of Array Queue

1. Simple implementation.
2. Efficient indexed storage.
3. No per-node pointer overhead.
4. Good memory locality.

Disadvantages of Array Queue

1. Fixed capacity unless dynamically resized.
2. Resizing requires copying elements.
3. A simple non-circular implementation can waste space.

Advantages of Linked Queue

1. Dynamic size.
2. No fixed array capacity.



3. Enqueue and dequeue can be O(1) with front and rear pointers.
4. Does not require contiguous memory.

Disadvantages of Linked Queue

1. Extra pointer memory.
2. Dynamic allocation overhead.
3. Allocation may fail.
4. Pointer management is more complex.

Time Complexity

Operation	Circular Array Queue	Linked Queue with FRONT and REAR
Enqueue	O(1)	O(1)
Dequeue	O(1)	O(1)
Peek	O(1)	O(1)
Display	O(n)	O(n)

Conclusion: Array queues are simple and memory-local, while linked queues provide flexible dynamic capacity. Both can perform basic queue operations in O(1) time when implemented appropriately.

MODULE 3

1. What is a Doubly Linked List? Implement its operations and explain with an example.

Definition

A Doubly Linked List (DLL) is a dynamic linear data structure in which each node contains three fields:

1. prev – pointer to the previous node.
2. data – stores the actual value.
3. next – pointer to the next node.

It allows traversal in both forward and backward directions.

Neat diagram



Structure of a node in C

```
struct node {
   int data;
   struct node *prev;
   struct node *next;
};
```

Operations of a doubly linked list

- Insertion at the beginning.
- Insertion at the end.
- Insertion at a specified position.

- Deletion at the beginning.
- Deletion at the end.
- Deletion of a specified node.
- Forward and backward traversal.

A. Insertion at the beginning

Algorithm

1. Create a new node.
2. Store the data in the new node.
3. Set newnode->prev = NULL.
4. Set newnode->next = head.
5. If the list is not empty, set head->prev = newnode.
6. Update head = newnode.

```
void insertBegin(struct node **head, int value) {
   struct node *newnode = malloc(sizeof(struct node));

   newnode->data = value;
   newnode->prev = NULL;
   newnode->next = *head;

   if (*head != NULL)
      (*head)->prev = newnode;

   *head = newnode;
}
```

B. Insertion at the end

Algorithm



1. Create a new node and store the data.
2. Set its next pointer to NULL.
3. If the list is empty, make it the head.
4. Otherwise, traverse to the last node.
5. Set the last node's next to the new node.
6. Set the new node's prev to the last node.

```
void insertEnd(struct node **head, int value) {
   struct node *newnode = malloc(sizeof(struct node));
   struct node *temp = *head;

   newnode->data = value;
   newnode->next = NULL;

   if (*head == NULL) {
      newnode->prev = NULL;
      *head = newnode;
      return;
   }

   while (temp->next != NULL)
      temp = temp->next;

   temp->next = newnode;
   newnode->prev = temp;
}
```

C. Deletion at the beginning



1. Check whether the list is empty.
2. Store the first node in a temporary pointer.
3. Move head to the second node.
4. If the new head exists, set its prev to NULL.
5. Free the old first node.

```
void deleteBegin(struct node **head) {
   if (*head == NULL)
      return;

   struct node *temp = *head;
   *head = (*head)->next;

   if (*head != NULL)
      (*head)->prev = NULL;

   free(temp);
}
```

D. Forward traversal

```
void display(struct node *head) {
   struct node *temp = head;

   while (temp != NULL) {
      printf("%d ", temp->data);
      temp = temp->next;
   }
}
```



Example

Insert 10, 20, and 30.

Before insertion

NULL ← 10 ⇄ 20 ⇄ 30 → NULL

Insert 5 at beginning

NULL ← 5 ⇄ 10 ⇄ 20 ⇄ 30 → NULL

Output

Forward: 5 10 20 30

Backward: 30 20 10 5

Advantages

1. Supports forward and backward traversal.
2. Insertion and deletion are efficient when the node position is known.
3. No need to shift elements as in arrays.
4. Useful for browser navigation, undo/redo, and playlists.

Disadvantages

1. Requires extra memory for the prev pointer.
2. More pointer updates than a singly linked list.
3. Implementation is more complex.
4. Searching still takes \(O(n)\) time.

Time complexity: Beginning insertion/deletion \(O(1)\); end insertion/deletion \(O(n)\) without a tail pointer; traversal \(O(n)\).

2. Define Circular Singly and Circular Doubly Linked Lists. Explain with a neat diagram.

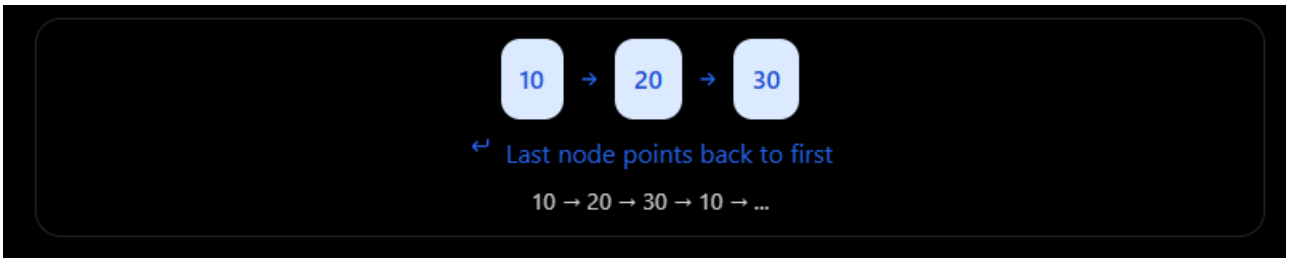
A. Circular Singly Linked List (CSLL)

Definition



A Circular Singly Linked List is a linked list in which every node contains data and a pointer to the next node. The last node points back to the first node instead of pointing to NULL.

Diagram



Characteristics

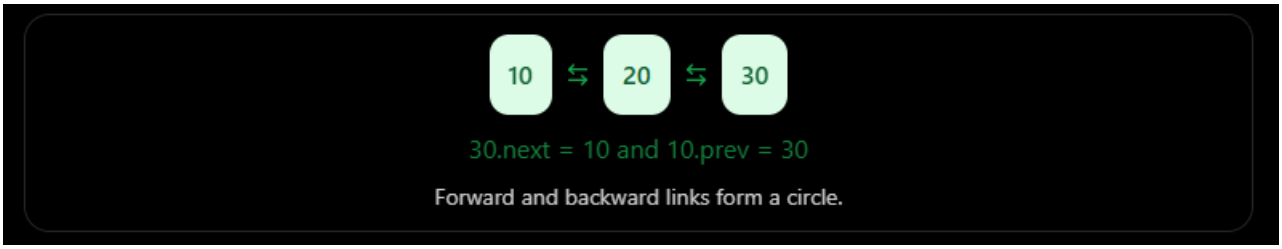
1. Last node's next points to the first node.
2. There is no NULL at the end.
3. Traversal can begin at any node.
4. Traversal must stop when the starting node is reached again.
5. Useful in round-robin scheduling and circular queues.

B. Circular Doubly Linked List (CDLL)

Definition

A Circular Doubly Linked List is a linked list in which every node contains prev, data, and next pointers. The last node connects to the first node, and the first node connects backward to the last node.

Diagram



Characteristics

1. Each node has three fields: prev, data, and next.
2. The last node's next points to the first node.

3. The first node's prev points to the last node.
4. Traversal is possible in both directions.
5. There is no NULL at either end.

Difference between CSLL and CDLL

Circular singly list	**Circular doubly list**
Two fields per node	Three fields per node
Has only next pointer	Has prev and next pointers
Forward traversal only	Forward and backward traversal
Less memory	More memory
Deletion needs previous-node tracking	Easier deletion if node pointer is available

3. Insert a node at the beginning and end of a Circular Singly Linked List

Definition

Insertion is the process of adding a new node to a linked list while maintaining its circular connection.

Assume we use a last pointer to refer to the last node. Then:

- last->next is the first node.
- If the list is empty, last = NULL.

Node structure

```
struct node {
  int data;
  struct node *next;
};
```

i) Insert at the beginning

Algorithm

1. Create a new node.



2. Store the given data.
3. If the list is empty:
    - Make the new node point to itself.
    - Set last = newnode.
4. Otherwise:
    - Set newnode->next = last->next.
    - Set last->next = newnode.
5. The new node becomes the first node.

C function

```
void insertBegin(struct node **last, int value) {
   struct node *newnode = malloc(sizeof(struct node));

   newnode->data = value;

   if (*last == NULL) {
      newnode->next = newnode;
      *last = newnode;
   } else {
      newnode->next = (*last)->next;
      (*last)->next = newnode;
   }
}
```

Example

Before

10 → 20 → 30 → back to 10

Insert 5 at beginning



5 → 10 → 20 → 30 → back to 5

ii) Insert at the end

Algorithm

1. Create a new node.
2. Store the given data.
3. If the list is empty:
    - Make the new node point to itself.
    - Set last = newnode.
4. Otherwise:
    - Set newnode->next = last->next.
    - Set last->next = newnode.
    - Update last = newnode.

C function

```
void insertEnd(struct node **last, int value) {
  struct node *newnode = malloc(sizeof(struct node));

  newnode->data = value;

  if (*last == NULL) {
    newnode->next = newnode;
    *last = newnode;
  } else {
    newnode->next = (*last)->next;
    (*last)->next = newnode;
    *last = newnode;
  }
```



}

Example

Before:

10 → 20 → 30 → back to 10

Insert 40 at end:

10 → 20 → 30 → 40 → back to 10

Time complexity: Both beginning and end insertion are \(O(1)\) when a last pointer is maintained.

4. Delete a node at the beginning and end of a Circular Singly Linked List

i) Delete at the beginning

Algorithm

1. If last == NULL, the list is empty.
2. Store the first node in temp = last->next.
3. If temp == last, only one node exists:
    - Set last = NULL.
4. Otherwise:
    - Set last->next = temp->next.
5. Free the deleted node.

C function

```
void deleteBegin(struct node **last) {
  if (*last == NULL) {
    printf("List is empty");
    return;
  }

  struct node *temp = (*last)->next;
```



```
  if (temp == *last) {
    *last = NULL;
  } else {
    (*last)->next = temp->next;
  }

  free(temp);
}
```

ii) Delete at the end

Algorithm

1. Check whether the list is empty.
2. If only one node exists:
    - Free it.
    - Set last = NULL.
3. Otherwise:
    - Traverse until the node before last is reached.
    - Set its next to the first node.
    - Update last to that node.
    - Free the old last node.

C function

```
void deleteEnd(struct node **last) {
  if (*last == NULL) {
    printf("List is empty");
    return;
  }
```



```
    struct node *temp = *last;

    if ((*last)->next == *last) {
        free(*last);
        *last = NULL;
        return;
    }

    struct node *prev = (*last)->next;

    while (prev->next != *last)
        prev = prev->next;

    prev->next = (*last)->next;
    *last = prev;

    free(temp);
}
```

Example

Initial circular list

10 → 20 → 30 → back to 10

Delete beginning (10)

20 → 30 → back to 20

Delete end (30)

20 → back to 20



Time complexity:

- Delete beginning: \(O(1)\)
- Delete end: \(O(n)\), because we must find the previous node.

5. Insert at the beginning and end of a Circular Doubly Linked List

Node structure

```
struct node {
   int data;
   struct node *prev;
   struct node *next;
};
```

Assume head points to the first node.

i) Insert at the beginning

Algorithm

1. Create a new node.
2. Store the given data.
3. If the list is empty:
    - Set newnode->next = newnode.
    - Set newnode->prev = newnode.
    - Set head = newnode.
4. Otherwise:
    - Find the last node using head->prev.
    - Set the new node's next to head.
    - Set its prev to the last node.
    - Update the last node's next.
    - Update the old head's prev.
    - Move head to the new node.



C function

```
void insertBegin(struct node **head, int value) {
   struct node *newnode = malloc(sizeof(struct node));

   newnode->data = value;

   if (*head == NULL) {
      newnode->next = newnode;
      newnode->prev = newnode;
      *head = newnode;
      return;
   }

   struct node *last = (*head)->prev;

   newnode->next = *head;
   newnode->prev = last;

   last->next = newnode;
   (*head)->prev = newnode;

   *head = newnode;
}
```

ii) Insert at the end

Algorithm

1. Create a new node.



2. If the list is empty, make it point to itself in both directions.
3. Otherwise:
    - Find the last node using head->prev.
    - Set the new node's next to head.
    - Set the new node's prev to the last node.
    - Set the last node's next to the new node.
    - Set head->prev to the new node.

C function

```
void insertEnd(struct node **head, int value) {
  struct node *newnode = malloc(sizeof(struct node));

  newnode->data = value;

  if (*head == NULL) {
    newnode->next = newnode;
    newnode->prev = newnode;
    *head = newnode;
    return;
  }

  struct node *last = (*head)->prev;

  newnode->next = *head;
  newnode->prev = last;

  last->next = newnode;
```



```
    (*head)->prev = newnode;
}
```

Example

Before

10 ⇄ 20 ⇄ 30 ⇄ back to 10

Insert 5 at beginning

5 ⇄ 10 ⇄ 20 ⇄ 30 ⇄ back to 5

Insert 40 at end

5 ⇄ 10 ⇄ 20 ⇄ 30 ⇄ 40 ⇄ back to 5

Time complexity: Both operations are \(O(1)\) when the head's prev points to the last node.

6. Delete at the beginning and end of a Circular Doubly Linked List

i) Delete at the beginning

Algorithm

1. If head == NULL, the list is empty.
2. Store the first node in temp.
3. If temp->next == temp, only one node exists:
    - Free it.
    - Set head = NULL.
4. Otherwise:
    - Find the last node using head->prev.
    - Move head to head->next.
    - Set the new head's prev to the last node.
    - Set the last node's next to the new head.
5. Free the old head.

C function



```
void deleteBegin(struct node **head) {
   if (*head == NULL) {
      printf("List is empty");
      return;
   }

   struct node *temp = *head;

   if (temp->next == temp) {
      free(temp);
      *head = NULL;
      return;
   }

   struct node *last = temp->prev;

   *head = temp->next;

   (*head)->prev = last;
   last->next = *head;

   free(temp);
}
```

ii) Delete at the end

Algorithm

1. Check whether the list is empty.



2. Store the last node in temp = head->prev.
3. If there is only one node:
    - Free it.
    - Set head = NULL.
4. Otherwise:
    - Find the node before the last node using temp->prev.
    - Set its next to head.
    - Set head->prev to that node.
5. Free the old last node.

C function

```
void deleteEnd(struct node **head) {
  if (*head == NULL) {
    printf("List is empty");
    return;
  }

  struct node *last = (*head)->prev;

  if (last == *head) {
    free(last);
    *head = NULL;
    return;
  }

  struct node *newlast = last->prev;
```



```
    newlast->next = *head;
    (*head)->prev = newlast;


    free(last);
}
```

Example

Initial:

10 ⇄ 20 ⇄ 30 ⇄ back to 10

- Delete beginning → 20 ⇄ 30 ⇄ back to 20
- Delete end → 20 ⇄ back to 20

Advantages of circular doubly linked lists

1. Traversal in both directions.
2. Efficient insertion and deletion at both ends.
3. No NULL at the ends.
4. Useful in circular navigation, playlists, and scheduling.

Disadvantages

1. Requires extra memory for prev.
2. Pointer updates must be done carefully.
3. Incorrect pointer handling may break the circular links.

7. What is a Polynomial? How is it represented using a Linked List? Give an example.

Definition

A polynomial is a mathematical expression consisting of variables, coefficients, and non-negative integer powers.

General form:

\[P(x)=a_nx^n+a_{n-1}x^{n-1}+\cdots+a_1x+a_0 \]



Where:

- \(a_n,a_{n-1},\ldots,a_0\) are coefficients.
- \(x\) is the variable.
- \(n\) is the exponent or degree.

Example

\[P(x)=5x^3+4x^2+2x+7 \]

Polynomial representation using a linked list

Each node contains three fields:

1. coeff – coefficient.
2. exp – exponent.
3. next – pointer to the next term.

C structure

```
struct poly {
   int coeff;
   int exp;
   struct poly *next;
};
```

Diagram

For the polynomial:

$$P(x) = 5x^3 + 4x^2 + 2x + 7$$



Each node stores one polynomial term.

Step-by-step representation

Coefficient	Exponent	Term
5	3	$5x^3$
4	2	$4x^2$
2	1	$2x$
7	0	7

Advantages

1. Stores only terms that exist.
2. Memory is allocated dynamically.
3. Easy to insert or remove polynomial terms.
4. Useful for polynomial addition and evaluation.

Disadvantages

1. Extra memory is required for pointers.
2. Accessing a particular term requires traversal.
3. Implementation is more complex than using an array.

8. Write algorithms to evaluate a polynomial and add two polynomials.

A. Polynomial evaluation

Definition

Polynomial evaluation means finding the value of a polynomial for a given value of \(x\).

Example:

\[P(x)=3x^2+2x+1 \]

For \(x=2\):

\[P(2)=3(2)^2+2(2)+1=17 \]

Algorithm: Evaluate polynomial using linked list

Input: Polynomial linked list and value of x.

1. Initialize result = 0.
2. Set temp = head.
3. While temp != NULL:
    - Calculate temp->coeff * pow(x, temp->exp).
    - Add the result to result.
    - Move to the next node.
4. Return result.

C function

```
#include <math.h>


int evaluate(struct poly *head, int x) {
  int result = 0;
  struct poly *temp = head;

  while (temp != NULL) {
    result += temp->coeff *
          pow(x, temp->exp);
```



```
    temp = temp->next;
  }

  return result;
}
```

Example

Polynomial:

$$ P(x)=3x^2+2x+1 $$

Given $x=2$:

Term	Calculation	Result
$3x^2$	$3(2^2)$	12
$2x$	$2(2)$	4
1	1	1
Total		17

B. Addition of two polynomials

Definition

Polynomial addition combines two polynomials by adding the coefficients of terms having the same exponent.

Example

$$ P(x)=5x^3+4x^2+2 $$

$$ Q(x)=3x^3+2x+7 $$

Adding:

$$ \boxed{R(x)=8x^3+4x^2+2x+9} $$

Algorithm



Assume both polynomial linked lists are arranged in descending exponent order.

1. Create an empty result list.
2. Set p to the first polynomial.
3. Set q to the second polynomial.
4. Compare their exponents:
    - If p->exp > q->exp, copy the p term.
    - If q->exp > p->exp, copy the q term.
    - If exponents are equal, add coefficients and store the result.
5. Move the corresponding pointer or both pointers.
6. Copy any remaining terms.
7. Return the result polynomial.

Pseudocode

POLY_ADD(P, Q)

1. Create empty result R

2. Set p = P and q = Q

3. While p != NULL and q != NULL:

    If p.exp > q.exp:

        Insert (p.coeff, p.exp) into R

        p = p.next

    Else if q.exp > p.exp:

        Insert (q.coeff, q.exp) into R

        q = q.next

Else:

sum = p.coeff + q.coeff

If sum != 0:

Insert (sum, p.exp) into R

p = p.next

q = q.next

4. Copy remaining terms of P into R

5. Copy remaining terms of Q into R

6. Return R

Step-by-step example

Polynomial P

$5x^3 + 4x^2 + 2$

Polynomial Q

$3x^3 + 2x + 7$

Add equal powers

$8x^3 + 4x^2 + 2x + 9$

Time complexity: \(O(m+n)\), where m and n are the numbers of terms in the two polynomials, assuming sorted exponents.

9. What is a Sparse Matrix? How is it represented using a Linked List? Give an example.

Definition



A Sparse Matrix is a matrix in which most of the elements are zero and only a small number of elements are non-zero.

Sparse matrices are used to save memory by storing only non-zero elements.

Example

Consider the matrix:

$$ A= \begin{bmatrix} 0&0&5&0\\ 0&8&0&0\\ 0&0&0&0\\ 3&0&0&6 \end{bmatrix} $$

This matrix has:

- Total elements = $4 \times 4 = 16$
- Non-zero elements = 4
- Zero elements = 12

Since most elements are zero, it is a sparse matrix.

Linked list representation

Each node stores:

1. row – row position of the non-zero element.
2. col – column position.
3. value – non-zero element.
4. next – pointer to the next node.

C structure

```
struct sparse {
   int row;
   int col;
   int value;
   struct sparse *next;
};
```

Neat diagram

Sparse matrix linked list



Step-by-step representation

Row	Column	Value
0	2	5
1	1	8
3	0	3
3	3	6

The zero elements are not stored.

Advantages

1. Saves memory.
2. Stores only non-zero elements.
3. Efficient for matrices with many zero values.
4. Useful in scientific computing and graph applications.

Disadvantages

1. Extra memory is needed for pointers.
2. Accessing an individual element may require traversal.
3. Less efficient for dense matrices.

10. Define Tree Data Structure. Explain Terminologies Associated with Tree. Give Example.

Definition

A Tree is a non-linear hierarchical data structure consisting of nodes connected by edges.

It represents data in a parent-child relationship.

A tree contains:

- One root node.
- Zero or more child nodes.
- Edges connecting nodes.
- No cycles.

Example

Tree structure

A

B

C

D

E

Tree terminologies

1. Node

An individual element in a tree is called a node.

Example: A, B, C, D and E.

2. Root

The topmost node of a tree is called the root.

Example: A.

3. Edge

A connection between two nodes is called an edge.

Example: A–B.



4. Parent

A node that has one or more children is called a parent.

Example: A is the parent of B and C.

5. Child

A node directly connected below another node is called its child.

Example: B and C are children of A.

6. Siblings

Nodes having the same parent are called siblings.

Example: B and C are siblings.

7. Leaf node

A node that has no children is called a leaf node.

Example: D and E.

8. Internal node

A node having at least one child is called an internal node.

Example: A and B.

9. Degree of a node

The number of children of a node is called its degree.

Example:

- Degree of A = 2
- Degree of B = 2
- Degree of D = 0

10. Degree of a tree

The maximum degree of any node in the tree is called the degree of the tree.

Example: Degree of this tree = 2.

11. Level

The level indicates the position of a node from the root.



Using root level 0:

- A → Level 0
- B, C → Level 1
- D, E → Level 2

12. Depth

The depth of a node is the number of edges from the root to that node.

Example: Depth of D = 2.

13. Height

The height of a tree is the maximum number of edges from the root to a leaf.

Example: Height = 2.

14. Subtree

A tree formed by a node and all its descendants is called a subtree.

Example: The subtree rooted at B contains B, D and E.

15. Ancestor

A node on the path from the root to a particular node is called its ancestor.

Example: A is an ancestor of D.

16. Descendant

A node below another node is called its descendant.

Example: D and E are descendants of A.

Applications of trees

1. File system directories.
2. Database indexing.
3. Organization hierarchy.
4. Expression evaluation.
5. Decision trees.
6. Searching and sorting.



11. Define Binary Tree. Explain Different Representations of Binary Tree.

Definition

A Binary Tree is a tree data structure in which each node can have at most two children:

1. Left child.
2. Right child.

The two children are called left and right subtrees.

Example

A

B

C

D

E

Representations of binary tree

There are two main representations:

1. Array representation.
2. Linked representation.

A. Array representation

In array representation, the nodes are stored in an array.

For a binary tree using 0-based indexing:

- Root is stored at index 0.
- Left child of index i is at 2*i + 1.
- Right child of index i is at 2*i + 2.
- Parent of index i is at (i-1)/2 using integer division, for i > 0.

Example

Binary tree

A is root; B and C are children; D and E are children of B.

Array representation

A

Index 0

B

Index 1

C

Index 2

D

Index 3

E

Index 4

Advantages

1. Simple representation.
2. Direct access to nodes by index.
3. No pointer storage required.
4. Efficient for complete binary trees.

Disadvantages

1. May waste memory for sparse trees.
2. Fixed-size arrays can limit growth.
3. Inserting nodes in irregular trees is inconvenient.

B. Linked representation

In linked representation, each node contains:

1. Data.
2. Pointer to left child.
3. Pointer to right child.

C structure

```
struct node {
   int data;
   struct node *left;
   struct node *right;
};
```

Diagram

data: A

left | right

B

C

Advantages

1. Dynamic memory allocation.
2. No unused array slots.
3. Easy insertion and deletion.
4. Suitable for irregular binary trees.

Disadvantages

1. Extra memory for pointers.
2. Requires dynamic memory allocation.
3. Pointer handling is more complex.

Difference between array and linked representation

Array representation	Linked representation
Uses an array	Uses nodes and pointers
Usually fixed capacity	Dynamic capacity
May waste space	Allocates space as needed
No pointer fields	Two child pointers per node



Good for complete trees Good for irregular trees

12. Discuss the Properties of Binary Tree

A binary tree has several important mathematical properties.

1. Maximum number of nodes at level

If the root is at level 0, the maximum number of nodes at level i is:

$$\boxed{2^i}$$

Example:

Level	Maximum nodes
0	1
1	2
2	4
3	8

2. Maximum number of nodes in a binary tree of height h

For height h, where height is measured in edges:

$$\boxed{N_{\max}=2^{h+1}-1}$$

Example: Height = 2.

$$N_{\max}=2^3-1=7$$

3. Minimum number of nodes for height h

For a binary tree with height h, the minimum number of nodes is:

$$\boxed{N_{\min}=h+1}$$

This occurs when the tree is skewed, with only one child at each level.

4. Number of leaf nodes in a perfect binary tree

A perfect binary tree of height h has:

$$\boxed{L=2^h}$$

leaf nodes.

Example: Height 3.

$$ L=2^3=8 $$

5. Number of internal nodes in a perfect binary tree

$$ \boxed{I=2^h-1} $$

Example: Height 3.

$$ I=2^3-1=7 $$

6. Relation between total nodes and height

For a binary tree of height h:

$$ h+1 \leq N \leq 2^{h+1}-1 $$

where N is the number of nodes.

7. Relation between leaf nodes and nodes with two children

In any non-empty binary tree:

$$ \boxed{n_0=n_2+1} $$

Where:

- n_0 = number of leaf nodes.
- n_2 = number of nodes having exactly two children.

8. Number of edges

For a tree containing n nodes:

$$ \boxed{E=n-1} $$

Example: If a tree has 7 nodes:

$$ E=7-1=6 $$

9. Maximum degree

The maximum degree of a node in a binary tree is:

$$ \boxed{2} $$

A node can have zero, one, or two children.

10. Number of NULL links



In a linked binary tree with n nodes, there are n + 1 NULL child pointers.

Each node has two child pointers, so total child pointers are 2n. A tree has n-1 actual edges.

\[NULL=2n-(n-1)=n+1 \]

Summary table

Property	Formula
Maximum nodes at level i	\(2^i\)
Maximum nodes at height h	\(2^{h+1}-1\)
Minimum nodes at height h	\(h+1\)
Leaves in perfect tree	\(2^h\)
Internal nodes in perfect tree	\(2^h-1\)
Edges in tree with n nodes	\(n-1\)
Leaf relation	\(n_0=n_2+1\)
NULL links in linked binary tree	\(n+1\)

Exam tip: For 8–10 marks, write the definition, draw a neat diagram, explain each property with its formula, and solve one small numerical example.